

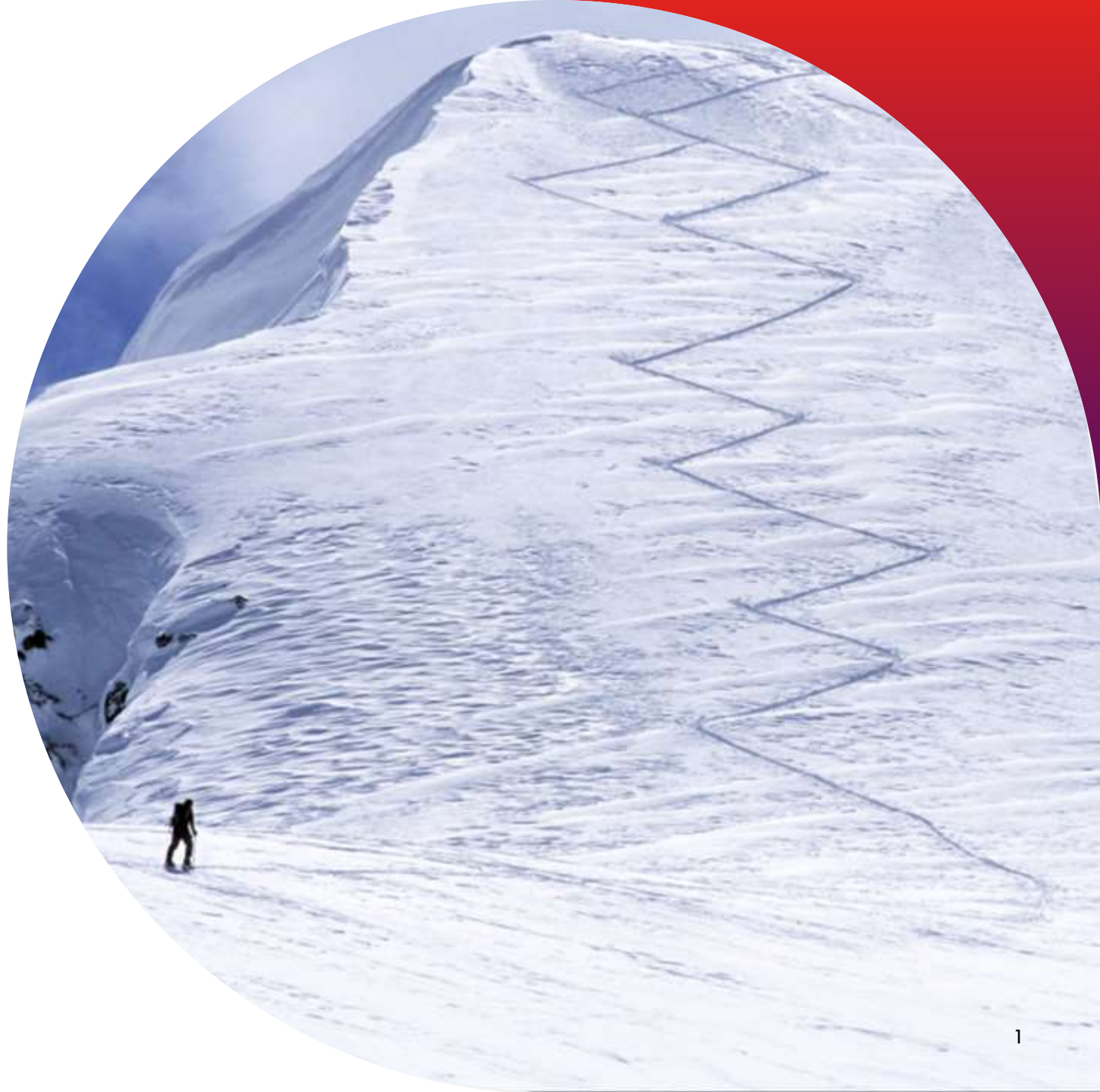
PARIS - CACHAN . TROYES . MONTPELLIER . SAINT-NAZAIRE . DAKAR

Numerical optimization for machine learning

T. Drumond



ÉCOLE D'INGÉNIEUR·E·S
Creating the future together



Class plan

Introduction (CM) : Optimization overview

General description of
an optimization
problem

Optimization
categories

Part I (CM+TD): Optimality conditions

Univariate and
multivariate conditions

Conditions for convex
functions

Conditions for
constrained optimization

Part II (CM+TD+TP): First order optimization algorithms

Gradient descent

Stochastic gradient
descent

Acceleration methods

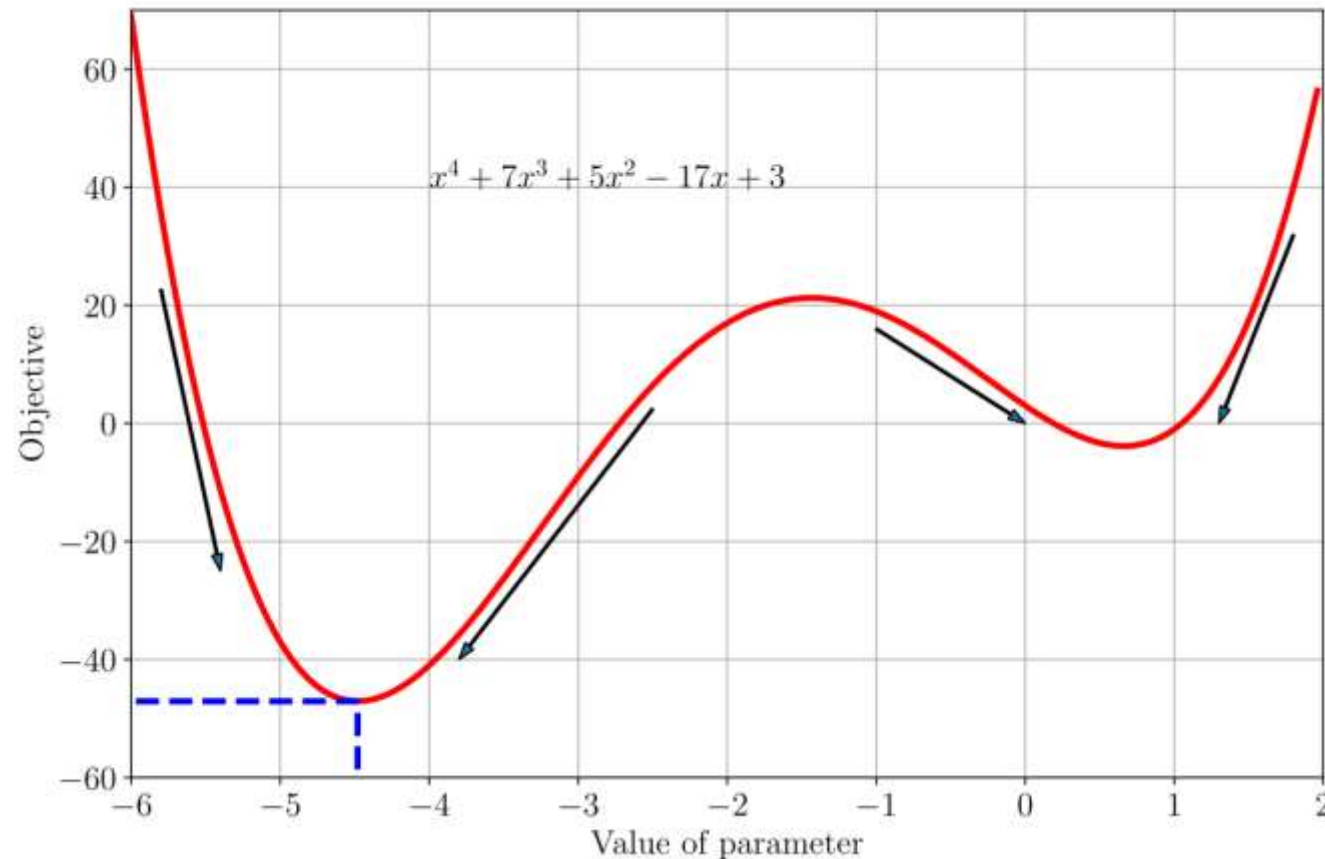
Part II

First order optimization algorithms

Optimization

Goal: Find parameters that minimize an objective function

Aka **cost function** or **loss function**



An **algorithm** that finds the optimal parameters for a given objective function is called a **solver**

Optimization algorithms

In general, we are unable to find analytic solutions

→ We need an iterative approach

Iterative descent algorithms

1. Start at some point x_0
2. Find out in which direction the cost function is decreasing
3. Take a step Δx in that direction
 - So that you have $x_1 = x_0 + \Delta x$
4. Repeat 2 and 3 until a minimum is reached

Questions

- **Where do we start?** Does it matter?
- How can we find the **direction of function decrease**?
- How do we know we have **reached a minimum**?
(or at least that we are close enough)
- **How large of a step** should we take?
- Will we **converge** to a local minimum? **How fast** will we get there?

Local vs global convergence

Problem: $\min_{\theta} \mathcal{L}(\theta)$

Update: $\theta_{t+1} = \theta_t + \Delta\theta_t$

We want the sequence

$$\mathcal{L}(\theta_0), \mathcal{L}(\theta_1), \dots, \mathcal{L}(\theta_t)$$

to converge to $\mathcal{L}(\theta_t) = \mathcal{L}(\theta^*)$ for a finite t

Local convergence

The sequence converges for initial points θ_0 in some vicinity of θ^* ,

i.e., for some $\delta > 0$, the sequence converges if θ_0 respects

$$\|\theta_0 - \theta^*\| \leq \delta$$

Global convergence

The sequence converges for any initial point θ_0 the domain of $\mathcal{L}(\theta)$

Convergence rate

Problem: $\min_{\theta} \mathcal{L}(\theta)$

Update : $\theta_{t+1} = \theta_t + \Delta\theta_t$

We want the sequence

$$\mathcal{L}(\theta_0), \mathcal{L}(\theta_1), \dots, \mathcal{L}(\theta_t)$$

to converge to $\mathcal{L}(\theta_t) = \mathcal{L}(\theta^*)$ for a finite t

Linear convergence

When there exists a number $0 < \mu < 1$ such that

$$|\mathcal{L}(\theta_{t+1}) - \mathcal{L}(\theta^*)| \leq \mu |\mathcal{L}(\theta_t) - \mathcal{L}(\theta^*)|$$

μ is called the **rate of convergence**

First-order vs second-order

First-order methods

Use only first-order (gradient) information to decide the stepping direction

Examples

- Gradient descent (GD)
- GD with momentum
- Nesterov accelerated gradient
- Stochastic GD (SGD)
- Preconditioned SGD variants: Adagrad, ADAM, RMSprop, etc ...

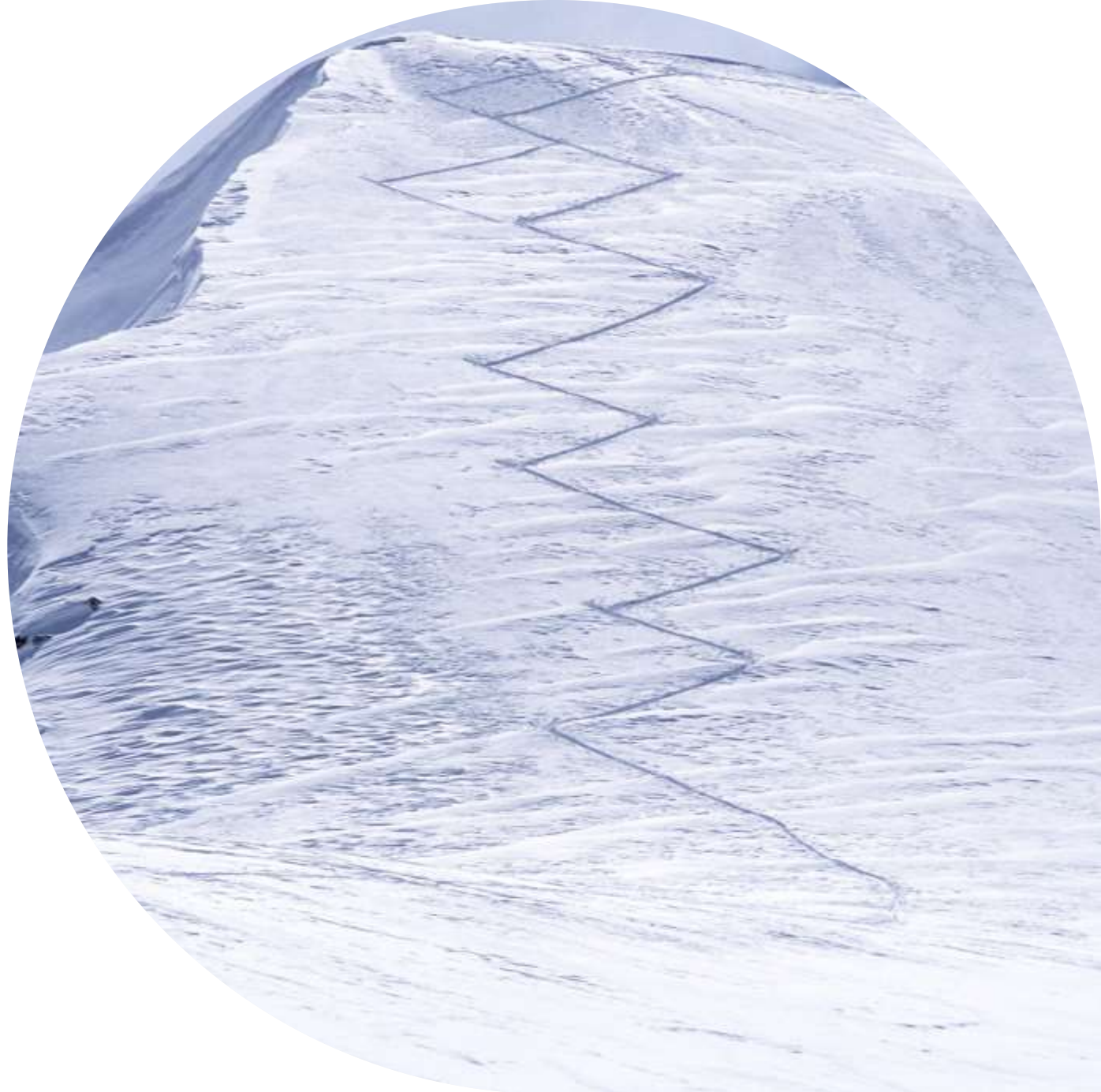
Second-order methods

Use second-order information (Hessian) to decide how large the step should be in each direction

Examples

- Newton's method
- BFGS and other quasi-Newton methods
- Trust region method

First-order algorithms
Gradient descent



First-order methods

Problem:

$$\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$$

Learning rate
or step size

Step:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \rho_t \mathbf{d}_t$$

Descent
direction

First-order methods

Descent direction

Definition

Any direction guaranteed to reduce cost for a certain step size $\rho < \rho_{max}$

$\mathbf{d}$ is a descent direction if there exists $\rho_{max} > 0$ such that

$$\mathcal{L}(\boldsymbol{\theta} + \rho \mathbf{d}) < \mathcal{L}(\boldsymbol{\theta})$$

for all $0 < \rho < \rho_{max}$

First-order methods

Step size

Any step $0 < \rho < \rho_{max}$ will decrease cost

Optimal step size

The optimal step ρ_t at iteration t is the one leading to the **smallest cost value** at $\theta_t + \rho_t d_t$
i.e.:

$$\rho_t = \operatorname{argmin}_{\rho > 0} \mathcal{L}(\theta_t + \rho d_t)$$

First-order methods

Gradient descent

Steepest descent = Gradient descent

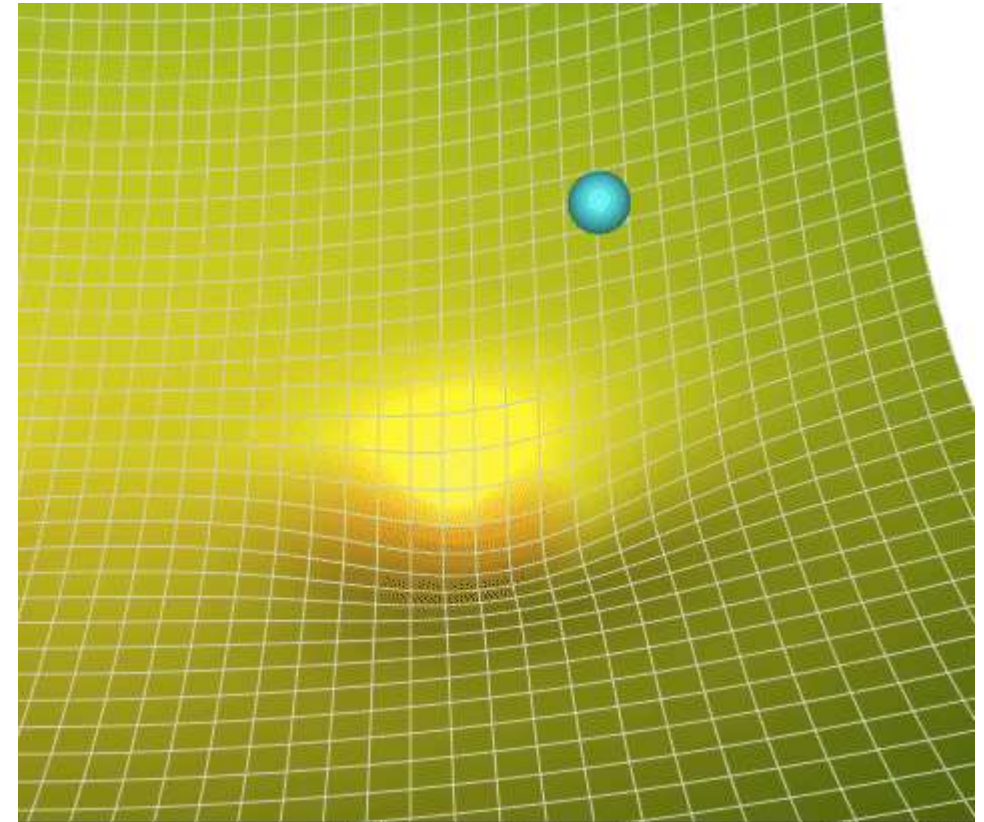
When we follow the opposite of the gradient

i.e. when we set the descent direction to

$$\mathbf{d}_t = -\nabla \mathcal{L}(\boldsymbol{\theta}) \Big|_{\boldsymbol{\theta}_t} = -\mathbf{g}(\boldsymbol{\theta}_t)$$

such that

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \rho_t \mathbf{g}(\boldsymbol{\theta}_t)$$



Gradient descent

Step size and convergence

Gradient descent step

$$\theta_{t+1} = \theta_t - \rho_t g(\theta_t)$$

For a sufficiently small step size ρ_t , we always decrease the cost function

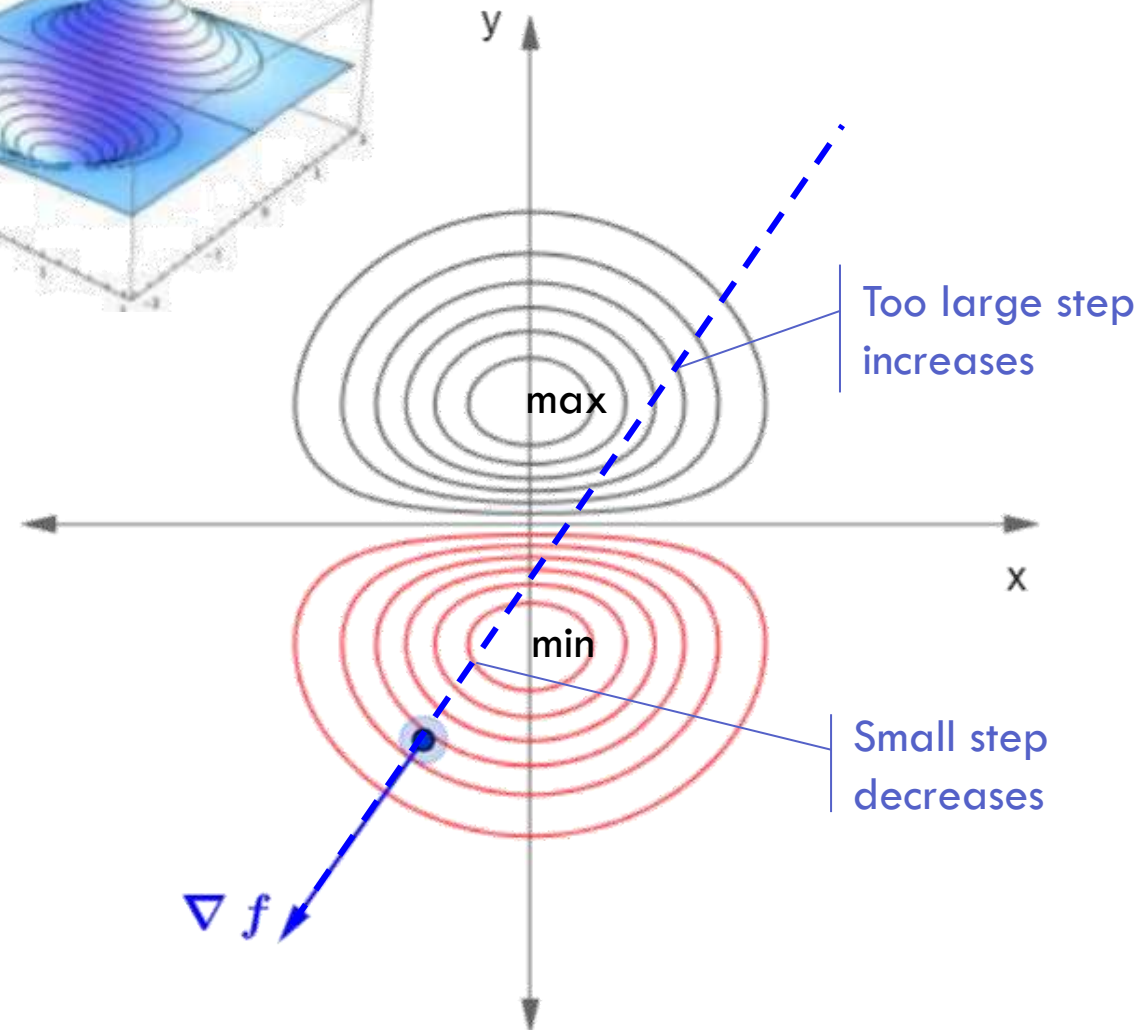
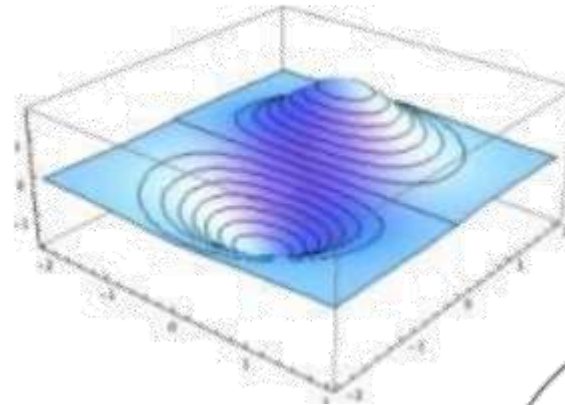
$$\mathcal{L}(\theta_t - \rho_t g(\theta_t)) < \mathcal{L}(\theta_t)$$

Note: If the function increases, i.e.,

$$\mathcal{L}(\theta_t - \rho_t g(\theta_t)) > \mathcal{L}(\theta_t)$$

then the step ρ_t was too large

$$f(x, y) = 4y \exp(-x^2 - y^2)$$



Gradient descent

Step size and convergence

Local convergence

We can say that for appropriate (not too large) step sizes ρ_t , the cost always decreases:

$$\mathcal{L}(\boldsymbol{\theta}_0) \geq \mathcal{L}(\boldsymbol{\theta}_1) \geq \dots \geq \mathcal{L}(\boldsymbol{\theta}_t)$$

We can prove that, for a nearby initial point and appropriate step sizes, gradient descent **will always converge to a local minimum**

For convex functions

GD converges for any given initial point



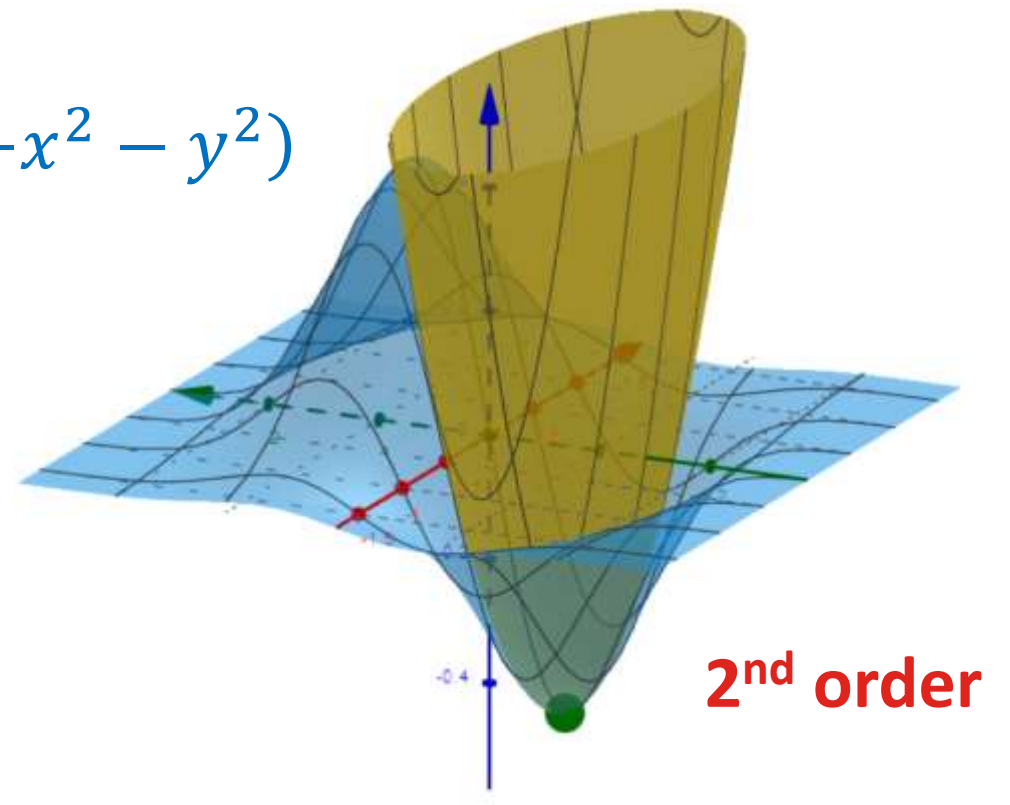
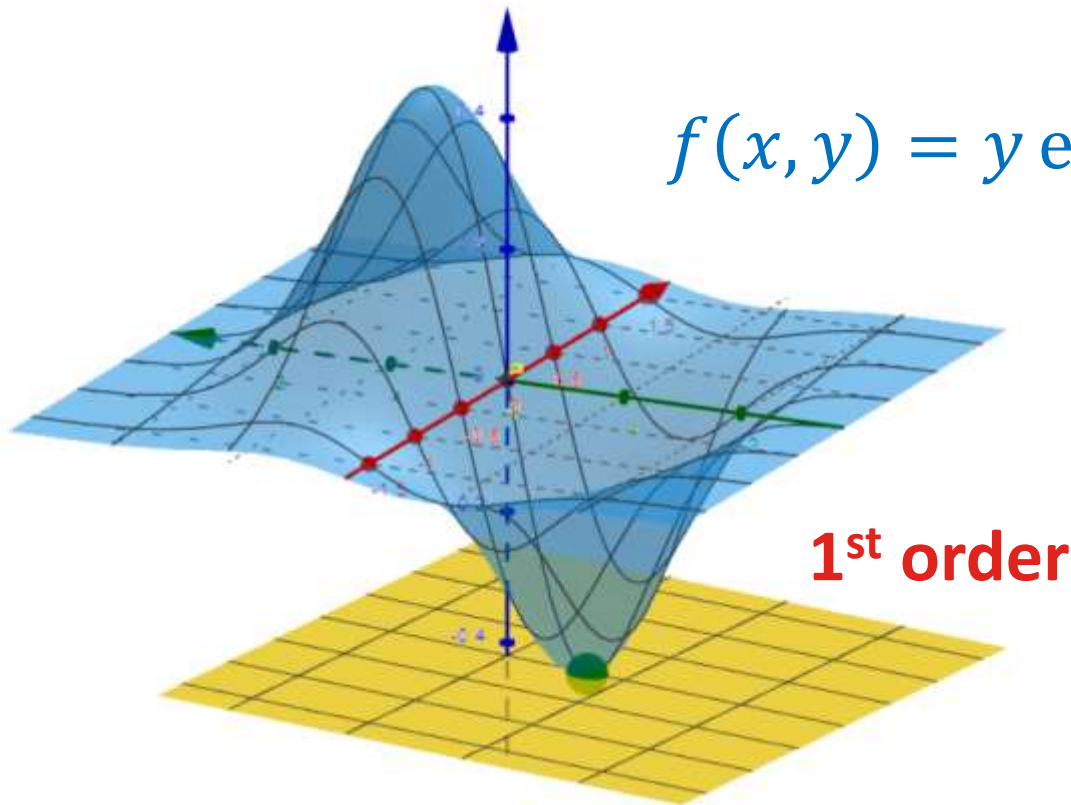
Gradient descent

Convergence

Second-order Taylor expansion of twice-differentiable $f: \mathbb{R}^n \rightarrow \mathbb{R}$ around $\mathbf{x}_0$:

$$f(\mathbf{x}) \approx f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0)^T (\mathbf{x} - \mathbf{x}_0) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_0)^T \nabla^2 f(\mathbf{x}_0) (\mathbf{x} - \mathbf{x}_0)$$

$$f(x, y) = y \exp(-x^2 - y^2)$$



Gradient descent

Convergence

Second-order Taylor expansion of twice-differentiable $f: \mathbb{R}^n \rightarrow \mathbb{R}$ around $\mathbf{x}_0$:

$$f(\mathbf{x}) \approx f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0)^T (\mathbf{x} - \mathbf{x}_0) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_0)^T \nabla^2 f(\mathbf{x}_0) (\mathbf{x} - \mathbf{x}_0)$$

Cost function approximation around a minimum

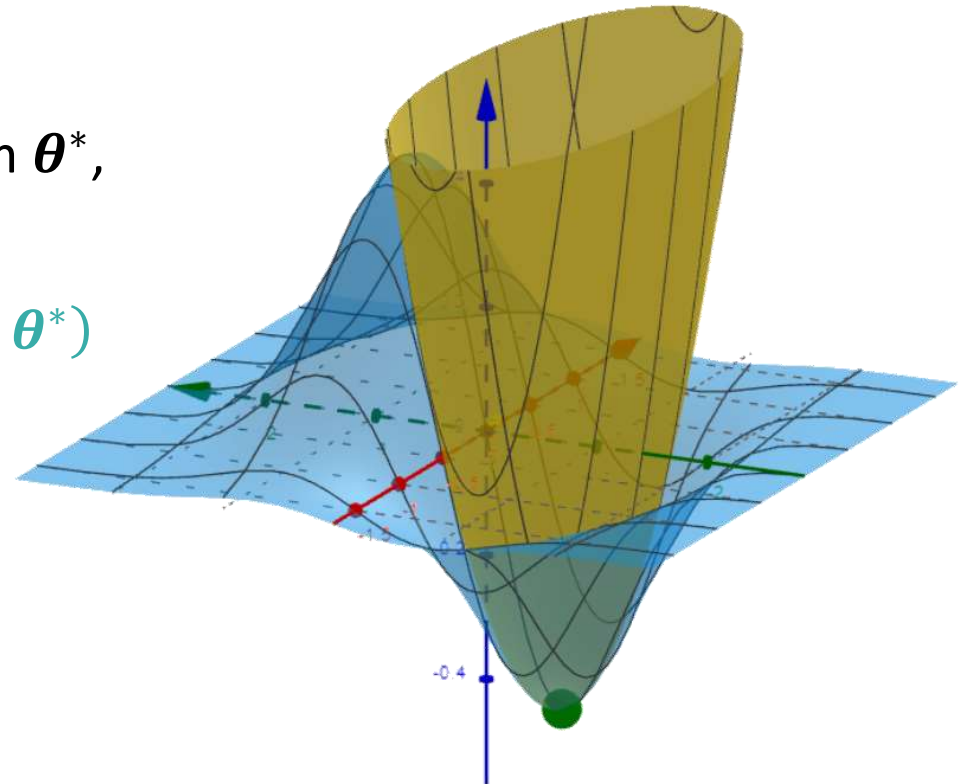
Given a certain cost function $\mathcal{L}(\boldsymbol{\theta})$ and a local minimum $\boldsymbol{\theta}^*$, we can approximate $\mathcal{L}(\boldsymbol{\theta})$ locally around $\boldsymbol{\theta}^*$ by

$$\mathcal{L}(\boldsymbol{\theta}^*) + g(\boldsymbol{\theta}^*)^T (\boldsymbol{\theta} - \boldsymbol{\theta}^*) + \frac{1}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}^*)^T \mathbf{H}_{\mathcal{L}}(\boldsymbol{\theta}^*) (\boldsymbol{\theta} - \boldsymbol{\theta}^*)$$

This approximation is **convex** since $\mathbf{H}_{\mathcal{L}}(\boldsymbol{\theta}^*) \succ 0$

Note: this approximation is a quadratic form:

$$c + \mathbf{b}^T \boldsymbol{\theta} + \frac{1}{2} \boldsymbol{\theta}^T \mathbf{A} \boldsymbol{\theta}$$



Gradient descent

Convergence on a quadratic cost

Convex quadratic cost

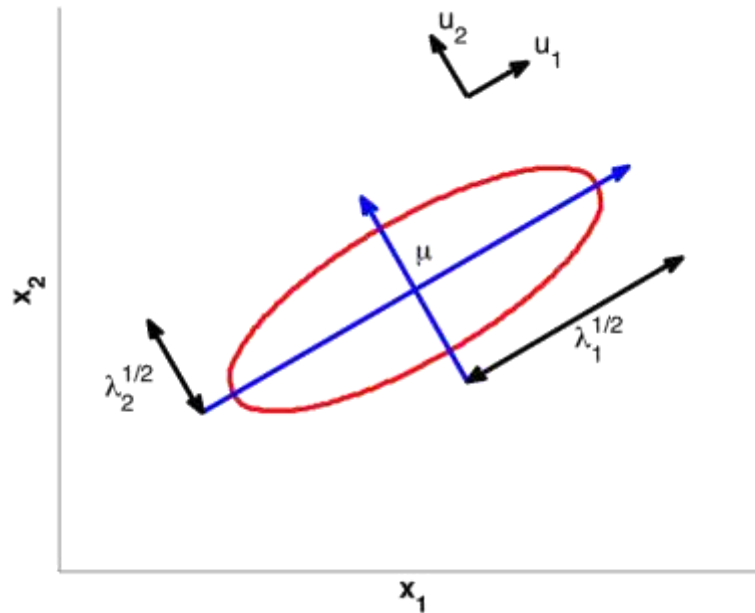
$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2} \boldsymbol{\theta}^T \mathbf{A} \boldsymbol{\theta} + \mathbf{b}^T \boldsymbol{\theta} + c$$

$\mathbf{A} \succcurlyeq 0$ (positive semidefinite)

Level sets

Ellipse along which cost value is constant

- Axis of the ellipse are oriented along $\mathbf{A}$'s eigenvectors
- Their length is proportional to $\mathbf{A}$'s eigenvalues



Level sets of a quadratic form $\mathbf{x}^T \mathbf{A} \mathbf{x}$: determined by $\mathbf{A}$'s eigenvalues and eigenvectors

Gradient descent

Convergence on a quadratic cost

Convex quadratic cost

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2} \boldsymbol{\theta}^T \mathbf{A} \boldsymbol{\theta} + \mathbf{b}^T \boldsymbol{\theta} + c$$

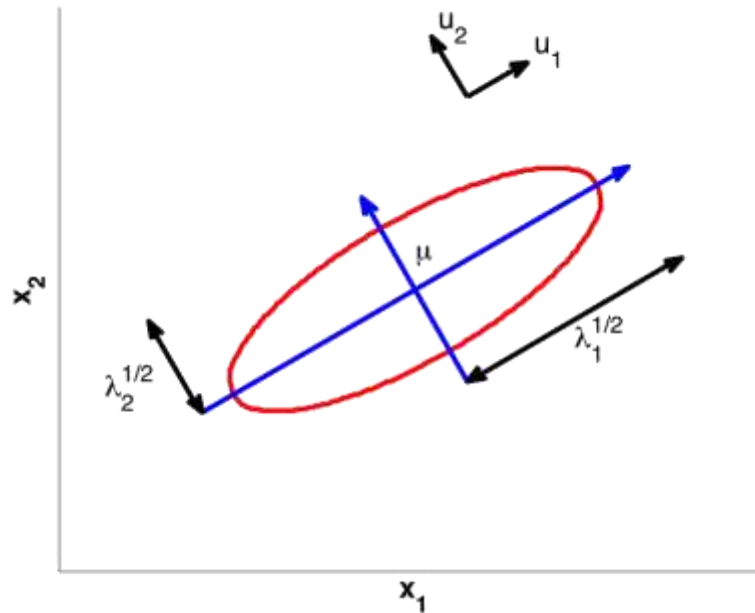
$\mathbf{A} \succcurlyeq 0$ (positive semidefinite)

Maximum step size

Gradient descent will have **global convergence** iff

$$\rho < \frac{2}{\lambda_{\max}(\mathbf{A})}$$

Where $\lambda_{\max}(\mathbf{A})$ is the largest eigenvalue of $\mathbf{A}$
 $-\lambda_{\max}(\mathbf{A})$ measures the slope of the steepest direction



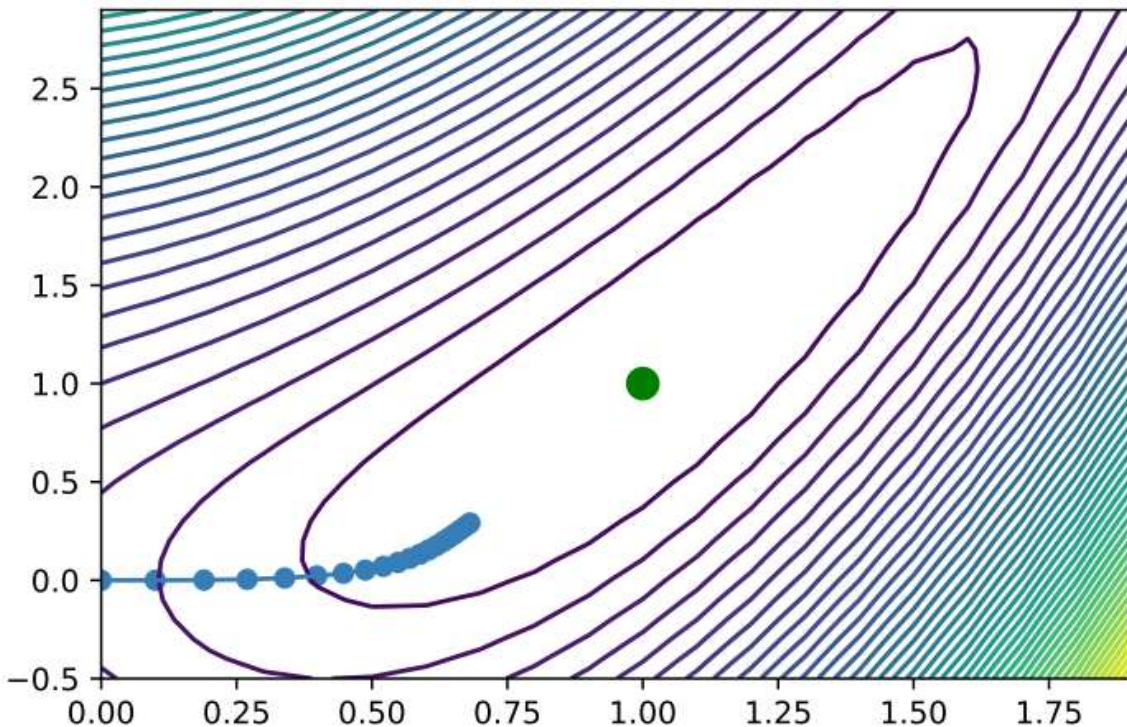
Level sets of a quadratic form $\mathbf{x}^T \mathbf{A} \mathbf{x}$: determined by $\mathbf{A}$'s eigenvalues and eigenvectors

Gradient descent

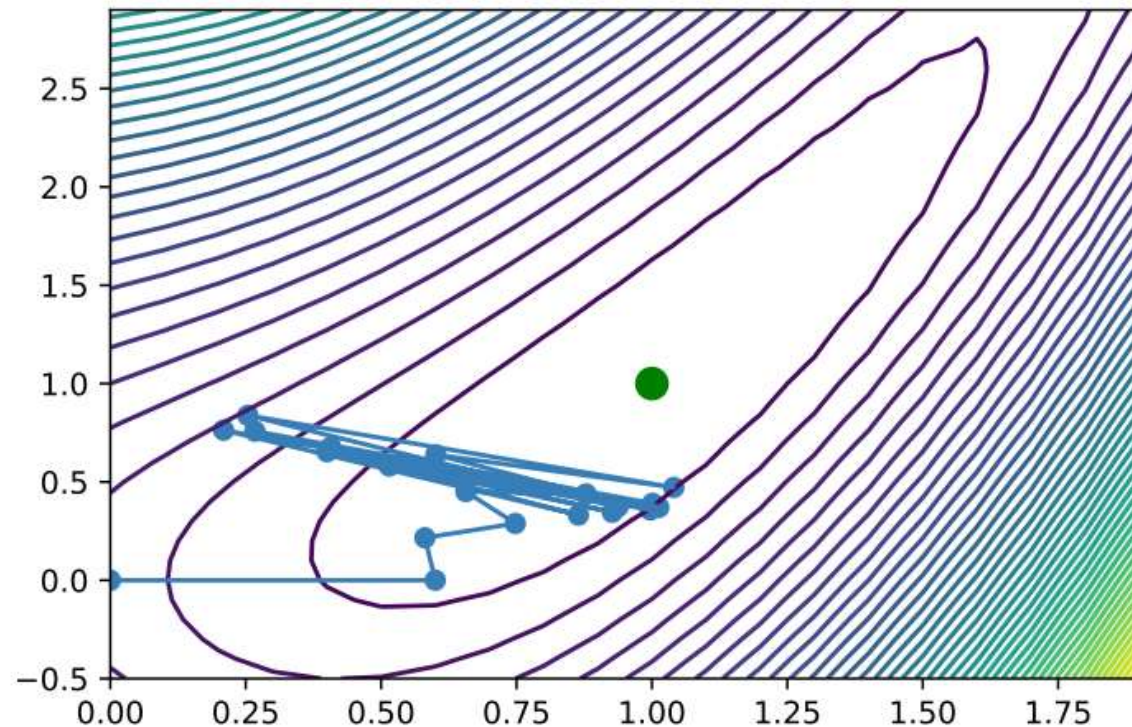
Convergence with constant step size

$$\mathcal{L}(\boldsymbol{\theta}) = 0.5(\theta_1^2 - \theta_2)^2 + 0.5(\theta_1 - 1)^2$$

step size 0.100



step size 0.600



About GD and convergence, select the true statements:

- ☒ 1 For a loss function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ that is linear on its parameters $\theta \in \mathbb{R}^n$, GD is guaranteed to converge to a local minimum for any given initial point and for a sufficiently small step size
- ☐ 2 For a loss function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ that is quadratic on its parameters $\theta \in \mathbb{R}^n$, GD is guaranteed to converge to a local minimum for any given initial point and for a sufficiently small step size
- ☐ 3 For a loss function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ that is convex and quadratic on its parameters $\theta \in \mathbb{R}^n$, GD is guaranteed to converge to a local minimum for any given initial point and for a sufficiently small step size
- ☐ 4 A succession of large steps can lead GD to diverge on a convex function

Gradient descent

Convergence on a quadratic cost

Convex quadratic cost

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2} \boldsymbol{\theta}^T \mathbf{A} \boldsymbol{\theta} + \mathbf{b}^T \boldsymbol{\theta} + c$$

$\mathbf{A} \succcurlyeq 0$ (positive semidefinite)

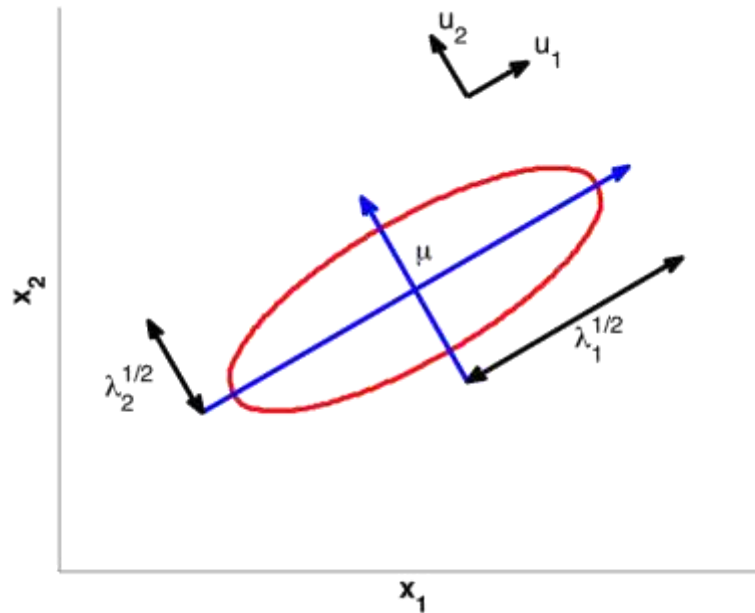
Optimal step size

Solving

$$\rho = \underset{\rho > 0}{\operatorname{argmin}} \mathcal{L}(\boldsymbol{\theta}_t + \rho \mathbf{d}_t)$$

gives the optimal step:

$$\rho_t = \frac{\mathbf{d}^T (\mathbf{A} \boldsymbol{\theta}_t + \mathbf{b})}{\mathbf{d}^T \mathbf{A} \mathbf{d}} = \frac{\mathbf{d}^T \mathbf{g}(\boldsymbol{\theta}_t)}{\mathbf{d}^T \mathbf{A} \mathbf{d}}$$



Level sets of a quadratic form $\mathbf{x}^T \mathbf{A} \mathbf{x}$: determined by $\mathbf{A}$'s eigenvalues and eigenvectors

Gradient descent

Convergence on a quadratic cost

Convex quadratic cost

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2} \boldsymbol{\theta}^T \mathbf{A} \boldsymbol{\theta} + \mathbf{b}^T \boldsymbol{\theta} + c$$

$\mathbf{A} \succcurlyeq 0$ (positive semidefinite)

Convergence rate at optimal step size

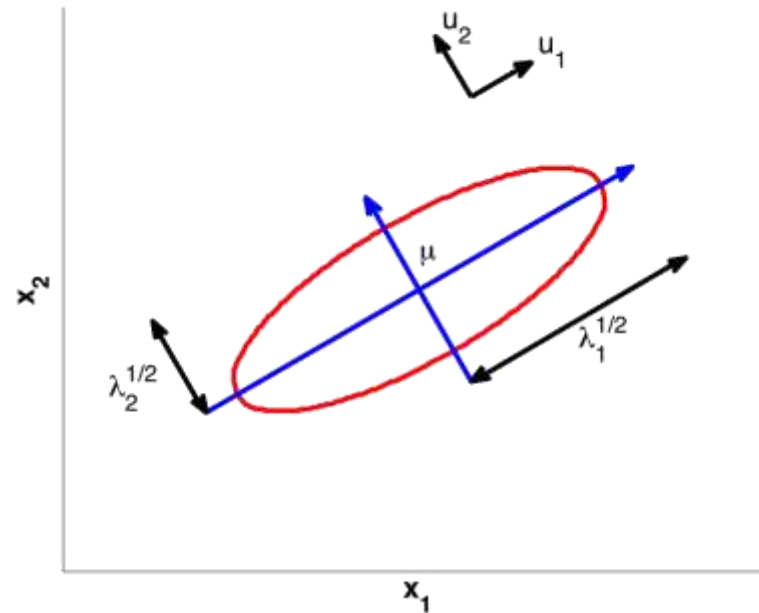
Linear rate of

$$\mu = \left(\frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} \right)^2 = \left(\frac{\kappa - 1}{\kappa + 1} \right)^2$$

where $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$ is the condition number of $\mathbf{A}$

Note $\kappa > 1$

κ close to 1 $\rightarrow \mu$ smaller (good 👍)



Level sets of a quadratic form $\mathbf{x}^T \mathbf{A} \mathbf{x}$: determined by $\mathbf{A}$'s eigenvalues and eigenvectors

Gradient descent

Convergence on a quadratic cost

Convex quadratic cost

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2} \boldsymbol{\theta}^T \mathbf{A} \boldsymbol{\theta} + \mathbf{b}^T \boldsymbol{\theta} + c$$

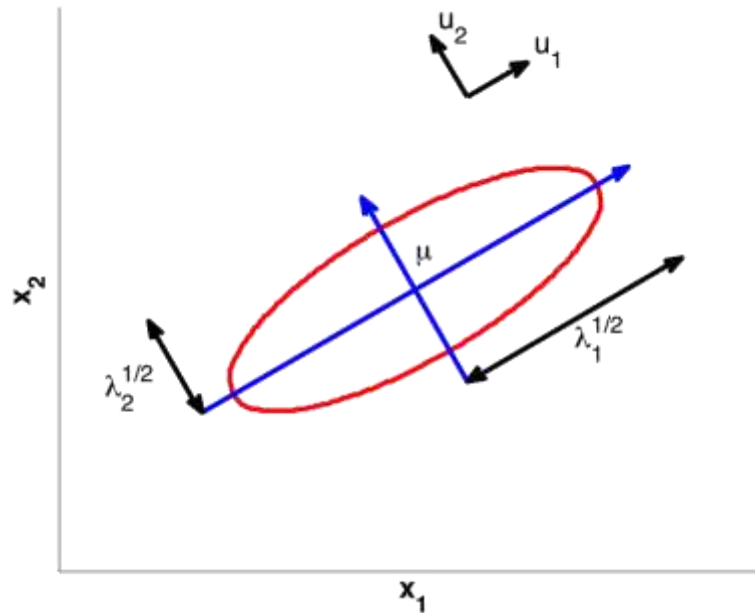
$\mathbf{A} \succcurlyeq 0$ (positive semidefinite)

Condition number

Ratio between max and min eigenvalues of $\mathbf{A}$

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$$

Measures how “skewed” the space is,
i.e., how far it is from a perfectly
symmetric bowl



Level sets of a
quadratic form $\mathbf{x}^T \mathbf{A} \mathbf{x}$:
determined by $\mathbf{A}$'s
eigenvalues and
eigenvectors

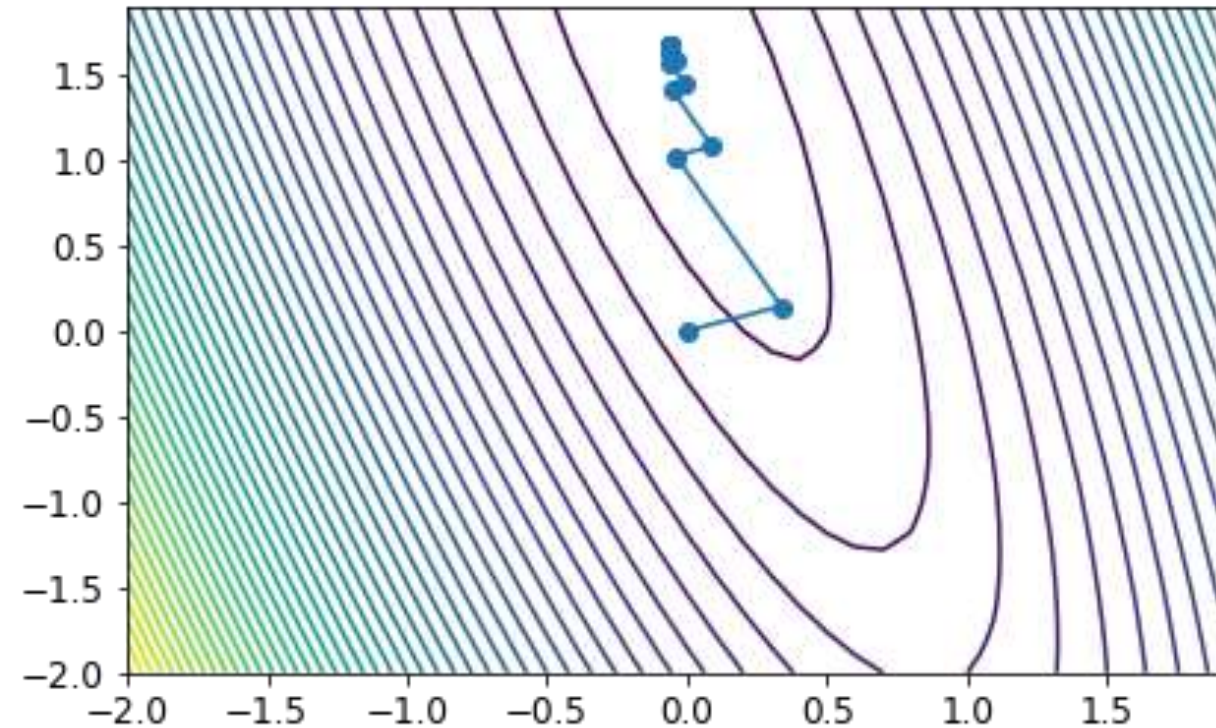
Gradient descent

Convergence with optimal step size

Large condition number κ

$$A = \begin{bmatrix} 20 & 5 \\ 5 & 2 \end{bmatrix}; b = \begin{bmatrix} -14 \\ -6 \end{bmatrix}; c = 10$$

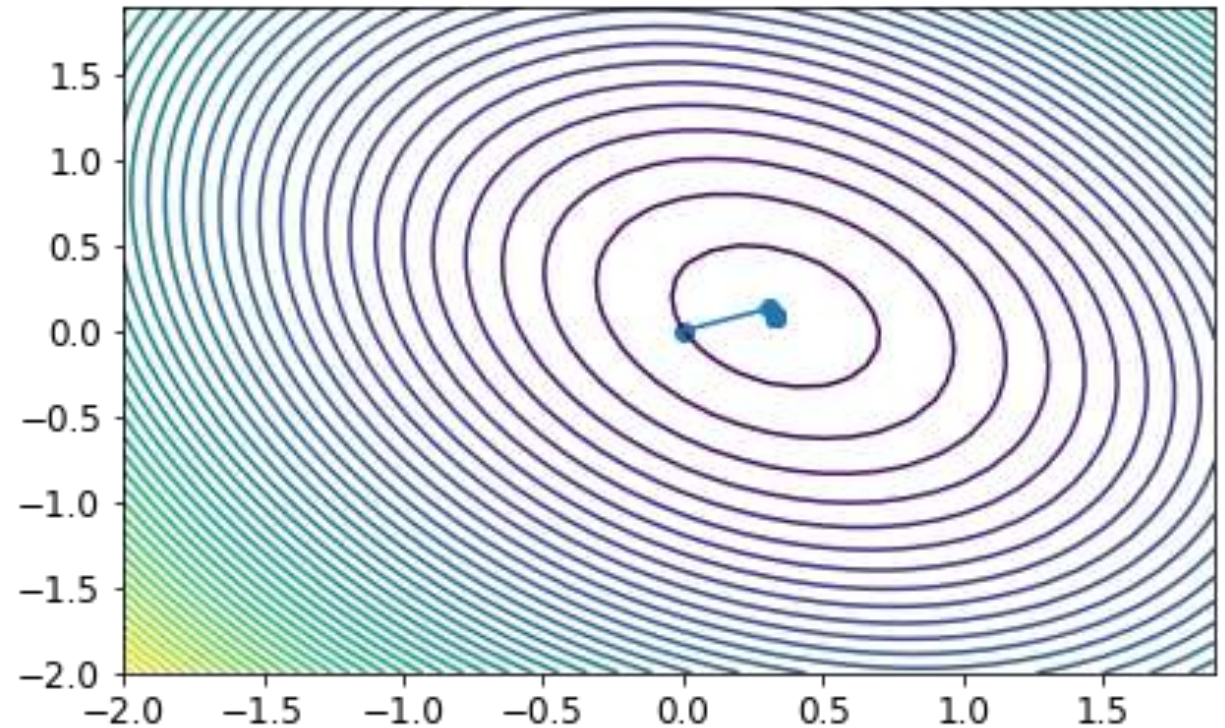
condition number of $A=30.234$



Small condition number κ

$$A = \begin{bmatrix} 20 & 5 \\ 5 & 16 \end{bmatrix}; b = \begin{bmatrix} -14 \\ -6 \end{bmatrix}; c = 10$$

condition number of $A=1.854$



About GD and the convex quadratic loss function, select the true statements:

- ① On such convex loss, a large step size can't lead GD to diverge
- ② Convergence rate is quadratic and depends on the Hessian of the loss function
- ③ Convergence rate is linear and depends on the condition number
- ④ For a convex quadratic loss function, convergence is faster when the max and min eigenvalues of the Hessian have similar values
- ⑤ A large hessian condition number at the minimum makes convergence faster

Click on the projected screen to start the question

Gradient descent

Convergence

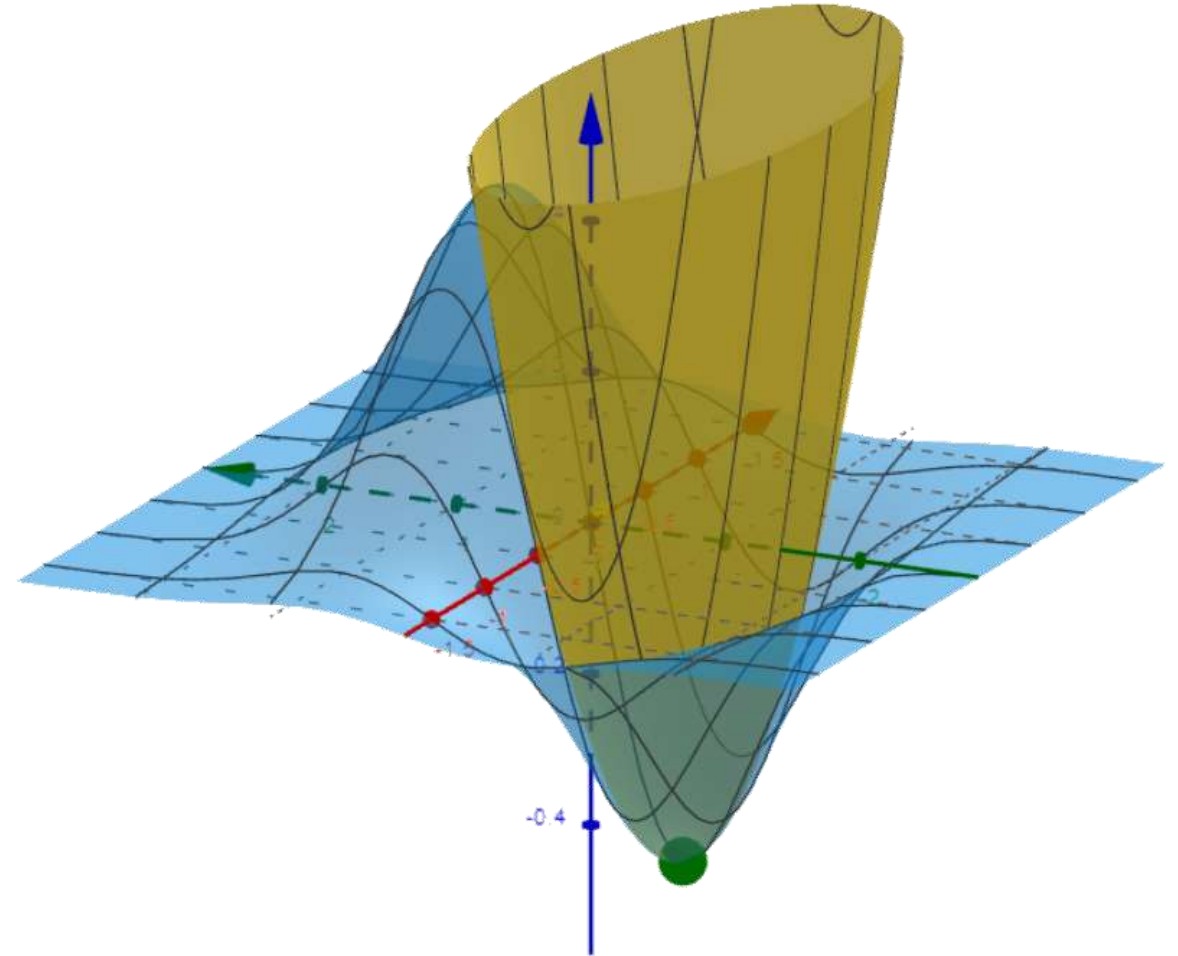
For non-quadratic functions

Remember the Taylor expansion:

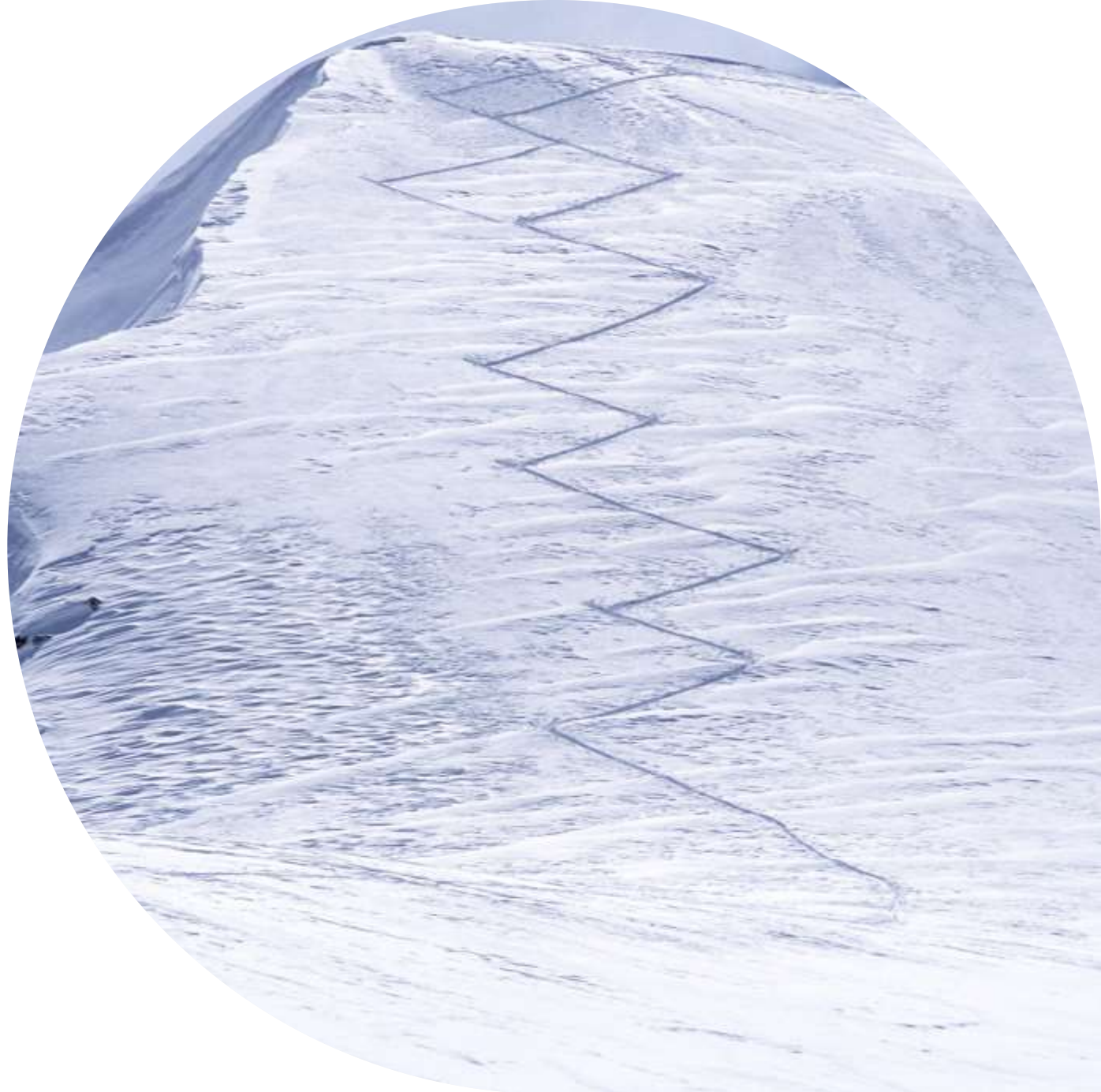
$$\begin{aligned} &\mathcal{L}(\boldsymbol{\theta}^*) + g(\boldsymbol{\theta}^*)^T(\boldsymbol{\theta} - \boldsymbol{\theta}^*) \\ &+ \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}^*)^T \mathbf{H}_{\mathcal{L}}(\boldsymbol{\theta}^*)(\boldsymbol{\theta} - \boldsymbol{\theta}^*) \end{aligned}$$

Twice-differentiable functions are **locally quadratic** for some small distance $\delta > 0$ around a local minimum

→ **Close to a minimum, convergence rate depends on the condition number $\kappa(\mathbf{H})$ for the Hessian matrix $\mathbf{H}$ at the minimum**



First-order algorithms
**Stochastic Gradient
descent**



Stochastic optimization

Context

$$\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}), \text{ where } \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}_n(\boldsymbol{\theta})$$

We can't observe $\mathcal{L}(\boldsymbol{\theta})$ **everywhere** ☹ (for all $n \in \{1, 2, \dots, N\}$)

But we can observe it at **some random locations** ☺ (for some $n \in \{1, 2, \dots, N\}$)

Original update step

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \rho_t \mathbf{g}_t = \boldsymbol{\theta}_t - \rho_t \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t)$$

Stochastic gradient descent

Note the original gradient is

$$\nabla_{\theta} \mathcal{L}(\theta_t) = \frac{1}{N} \sum_{n=1}^N \nabla_{\theta} \mathcal{L}_n(\theta)$$

Instead of $\nabla_{\theta} \mathcal{L}(\theta_t)$ we will use the following estimate

$$\hat{g}_t = \nabla_{\theta} \mathcal{L}_n(\theta_t)$$

where n is a different random observation at each iteration t .

The update step then becomes

$$\theta_{t+1} = \theta_t - \rho_t \hat{g}_t = \theta_t - \rho_t \nabla_{\theta} \mathcal{L}_n(\theta_t)$$

Convergence

If the gradient estimate $\hat{g}_t$ is **unbiased** $\rightarrow$ SGD will converge to a **stationary point**

Stochastic gradient descent

Typical machine learning case

Cost:

Empirical risk: average of individual costs $\mathcal{L}_n(\boldsymbol{\theta}_t)$ per sample $(\mathbf{x}_n, \mathbf{y}_n)$

$$\mathcal{L}(\boldsymbol{\theta}_t) = \frac{1}{N} \sum_{n=1}^N \ell(\mathbf{y}_n, f(\mathbf{x}_n; \boldsymbol{\theta}_t)) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}_n(\boldsymbol{\theta}_t)$$

Full-batch gradient computation

Average of the gradients for each sample $(\mathbf{x}_n, \mathbf{y}_n)$

$$\mathbf{g}_t = \frac{1}{N} \sum_{n=1}^N \nabla_{\boldsymbol{\theta}} \mathcal{L}_n(\boldsymbol{\theta}_t) = \frac{1}{N} \sum_{n=1}^N \nabla \ell(\mathbf{y}_n, f(\mathbf{x}_n; \boldsymbol{\theta}_t))$$

Gradient computation **uses all available samples**

Stochastic gradient descent

Typical machine learning case

Full-batch gradient computation

Average of the gradients for each sample $(\mathbf{x}_n, \mathbf{y}_n)$

$$\mathbf{g}_t = \frac{1}{N} \sum_{n=1}^N \nabla_{\boldsymbol{\theta}} \mathcal{L}_n(\boldsymbol{\theta}_t) = \frac{1}{N} \sum_{n=1}^N \nabla \ell(\mathbf{y}_n, f(\mathbf{x}_n; \boldsymbol{\theta}_t))$$

Gradient computation **uses all available samples**

Stochastic gradient estimate

Take **one random sample** $(\mathbf{x}_n, \mathbf{y}_n)$ and calculate the gradient over it

$$\mathbf{g}_t \approx \nabla_{\boldsymbol{\theta}} \mathcal{L}_n(\boldsymbol{\theta}_t) = \nabla \ell(\mathbf{y}_n, f(\mathbf{x}_n; \boldsymbol{\theta}_t))$$

Disadvantage: $\mathbf{g}_t$ has **high variance** (it is very noisy)

Stochastic gradient descent

Mini-batch SGD

Idea

Compromise between full-batch GD and single sample SGD

Reduce variance in $\mathbf{g}_t$ by averaging over a pool of samples

Mini-batch stochastic gradient estimate

Take a **subset of samples** of size $B \ll N$ and average the gradients over it

$$\mathbf{g}_t \approx \frac{1}{B} \sum_{n=1}^B \nabla_{\theta} \mathcal{L}_n(\boldsymbol{\theta}_t) = \frac{1}{B} \sum_{n=1}^B \nabla \ell(\mathbf{y}_n, f(\mathbf{x}_n; \boldsymbol{\theta}_t))$$

Convergence

Also provides an unbiased estimate of the gradient

→ will also converge to a stationary point

Stochastic gradient descent

SGD vs GD

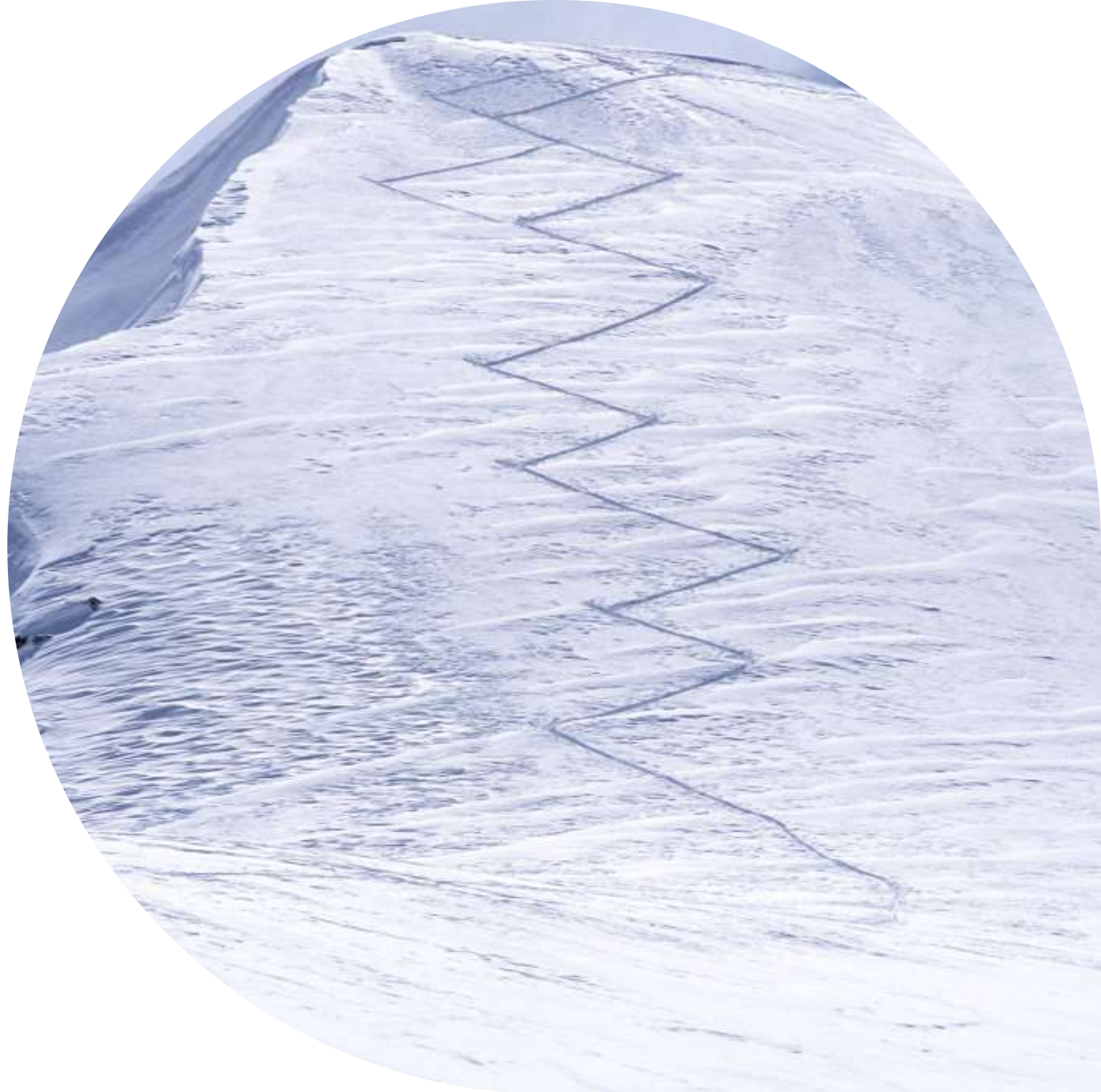
Convergence in theory

- GD: local convergence at **linear** rate
- SGD: convergence to stationary point at **sublinear** rate

In practice

- GD iterations are slower than SGD for large datasets → SGD faster overall
- Mini-batch SGD is more computationally efficient than GD

First-order algorithms
GD limitations



Go to **wooclap.com** and use the code **GDNCO**



Let's vote!



0

Click on the projected screen to start the question

answers received



wooclap



120%



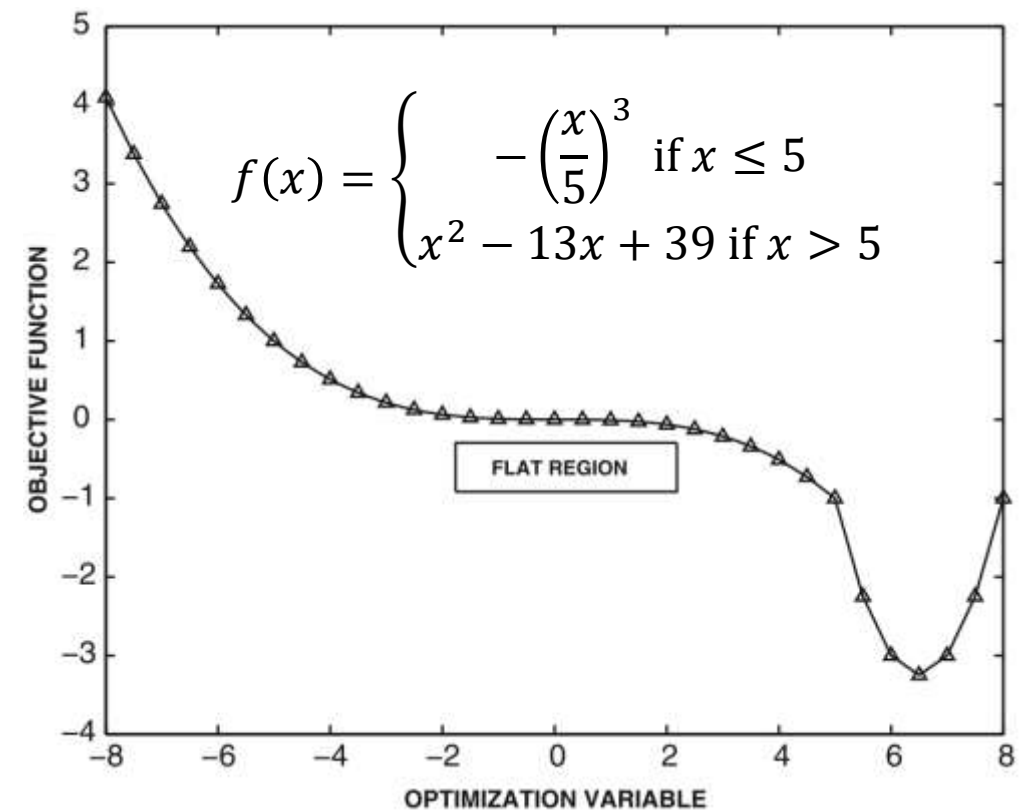
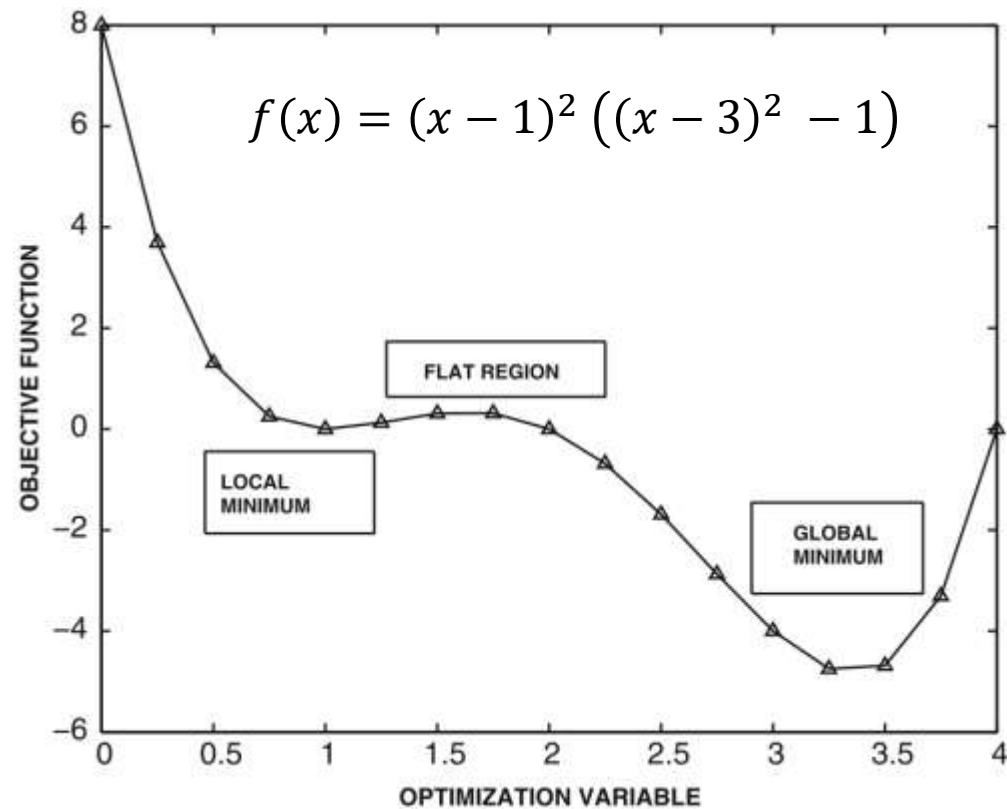
0



Gradient descent limitations

Local optima and flat regions

Where gradient is small $\rightarrow$ convergence slows down



Gradient descent limitations

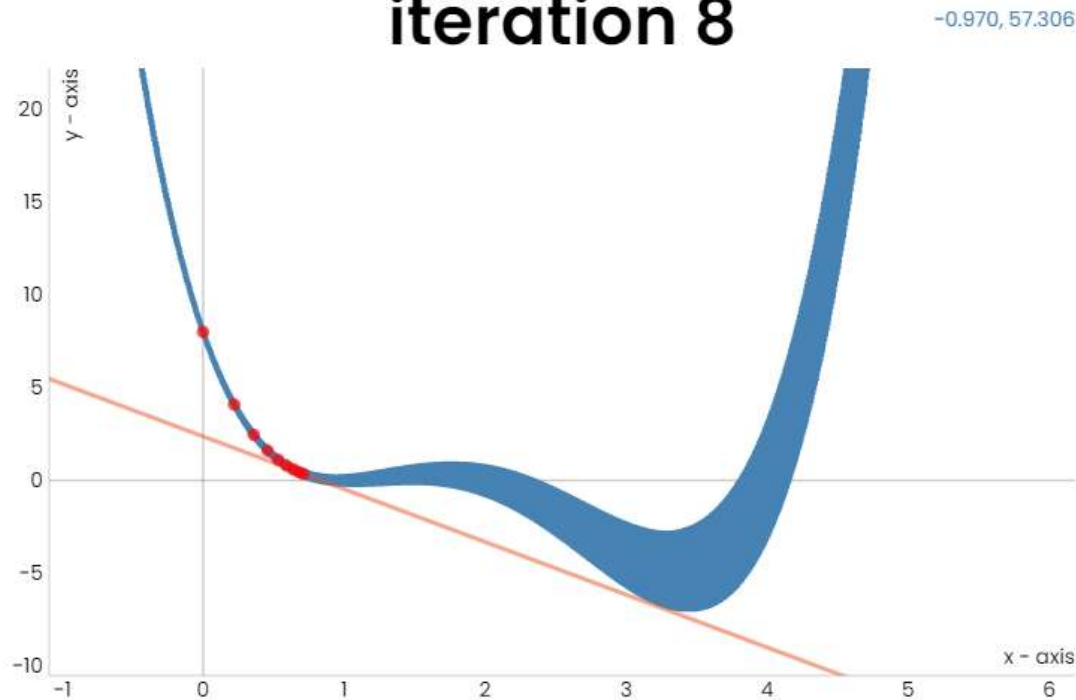
Local optima and flat regions

GD may get stuck at bad local minima

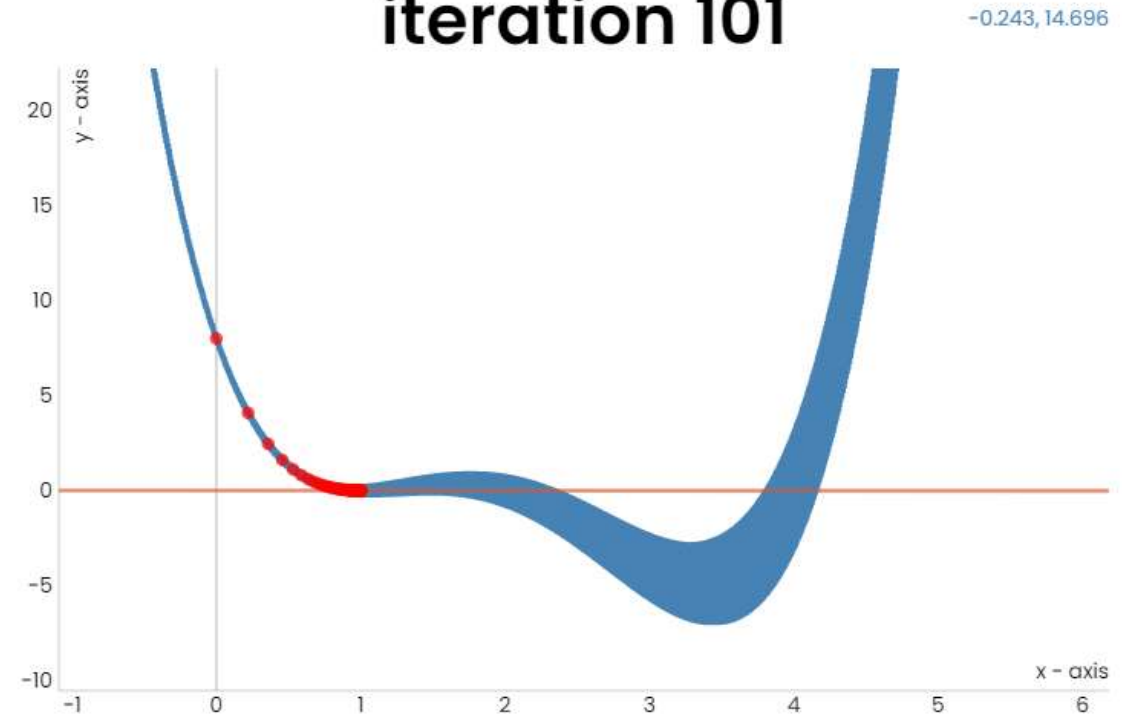
Example:

$f(x) = (x - 1)^2 ((x - 3)^2 - 1)$, step size=0.01, start at x=0

iteration 8



iteration 101



Gradient descent limitations

Local optima and flat regions

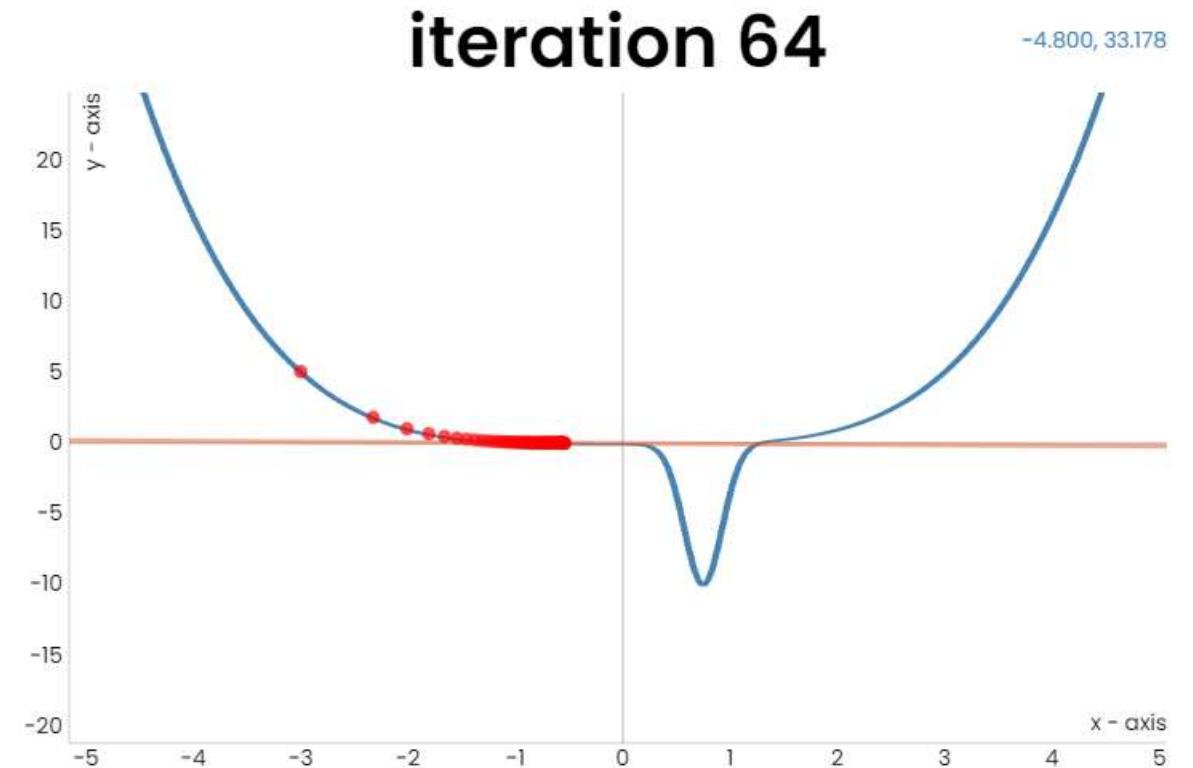
GD descends very slowly at flat regions

Example:

$$f(x) = \left(\frac{x}{2}\right)^4 - 10 \exp(-4x - 3)^2$$

start at $x=-3$

step size=0.1 → Stuck at a plateau



Gradient descent limitations

Local optima and flat regions

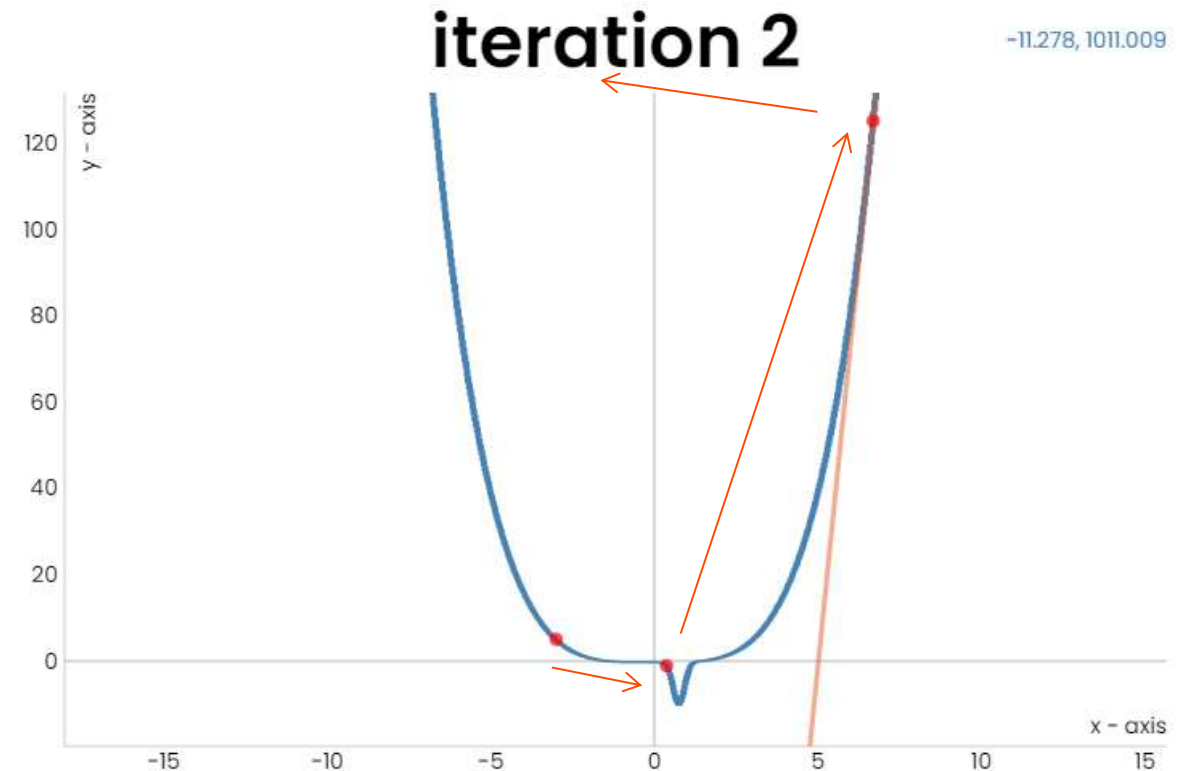
Example:

Same function, same initial point, **higher step size**

$$f(x) = \left(\frac{x}{2}\right)^4 - 10 \exp(-4x - 3)^2$$

start at $x=-3$

step size=0.5 → GD overshoots the minimum and diverges



Gradient descent limitations

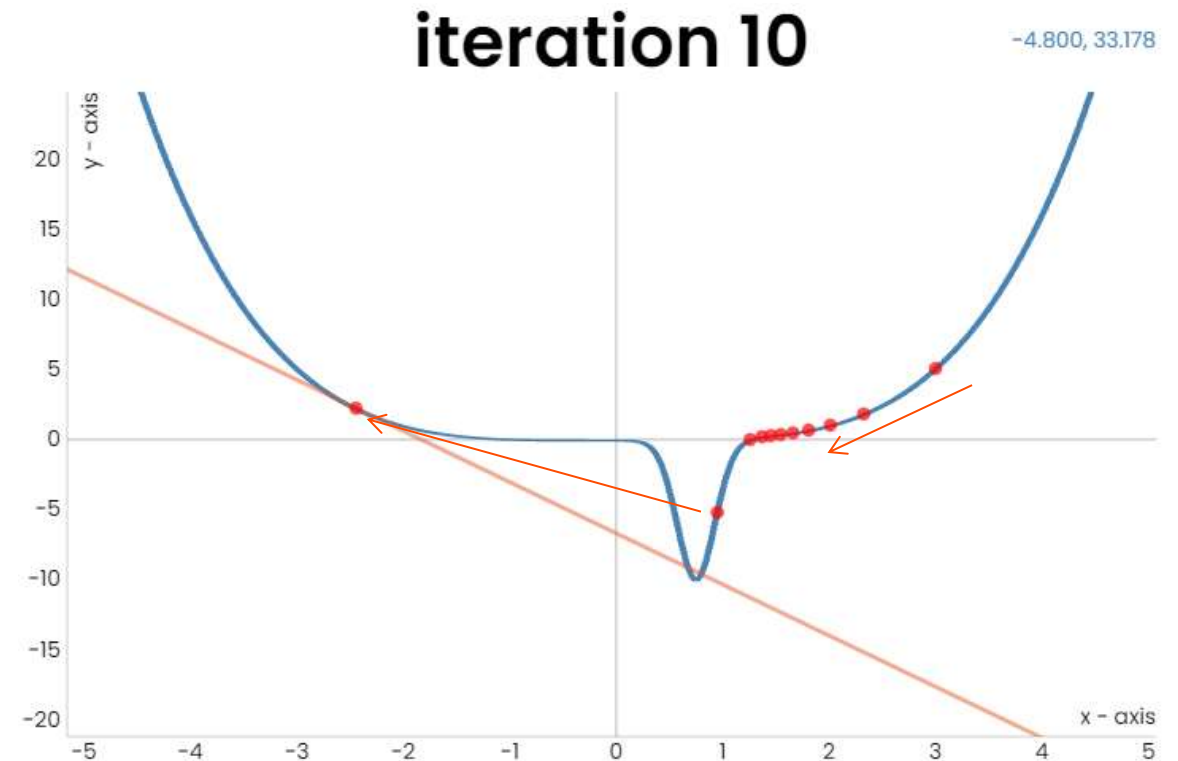
Local optima and flat regions

GD may miss sharp minima

Example:

$$f(x) = \left(\frac{x}{2}\right)^4 - 10 \exp(-4x - 3)^2$$

start at $x=3$, step size=0.1

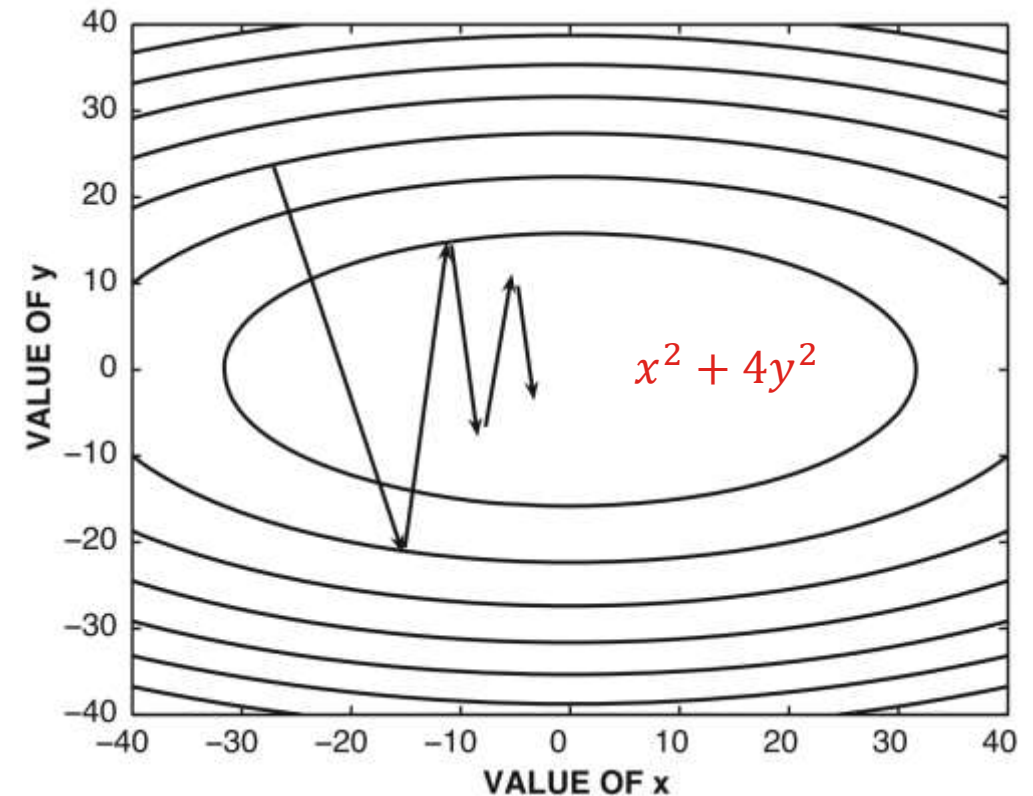
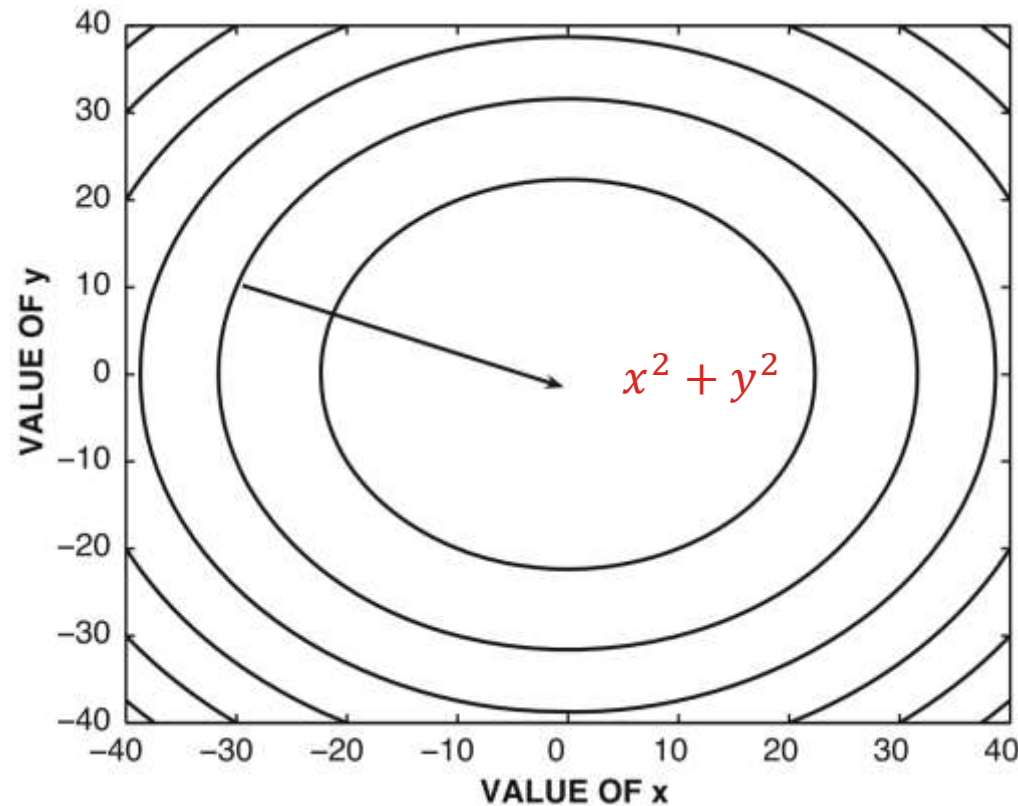


Gradient descent limitations

Differential curvature

GD takes much longer to converge when curvatures across different directions are too different

Remember the condition number for the quadratic loss



Gradient descent limitations

Differential curvature

GD takes much longer to converge when curvatures across different directions are too different

Remember the condition number for the quadratic loss

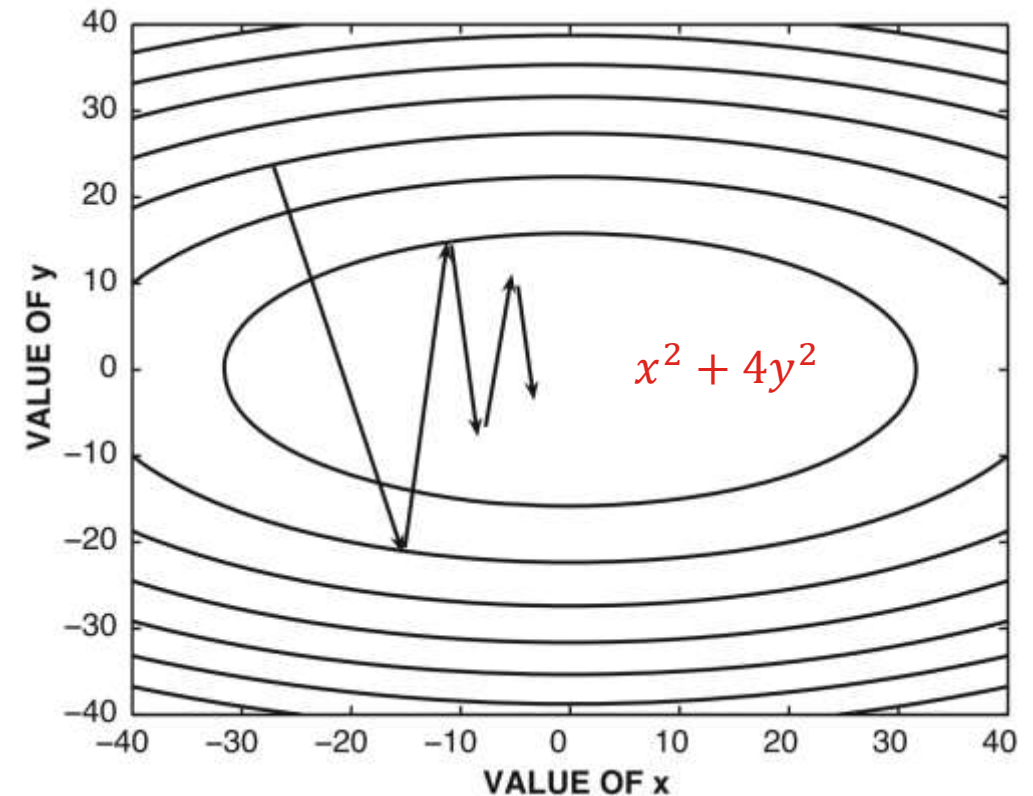
Movement along the Y axis is inefficient

Y steps are large but in zig-zag

X steps are small but consistent

GD step never points directly to the minimum

The steepest descent is only an instantaneous best movement, not the best movement overall



We'd like methods that

avoid zigzagging on
excentric curvatures

move faster along
flat regions

Improving gradient descent

We'd like methods that

avoid zigzagging on excentric
curvatures

move faster along flat regions

Momentum methods

Preconditioned GD methods

Simple
momentum

Nesterov
acceleration

Adam

AdaGrad

RMSprop

First-order algorithms

Accelerating GD with
momentum



Gradient descent with momentum

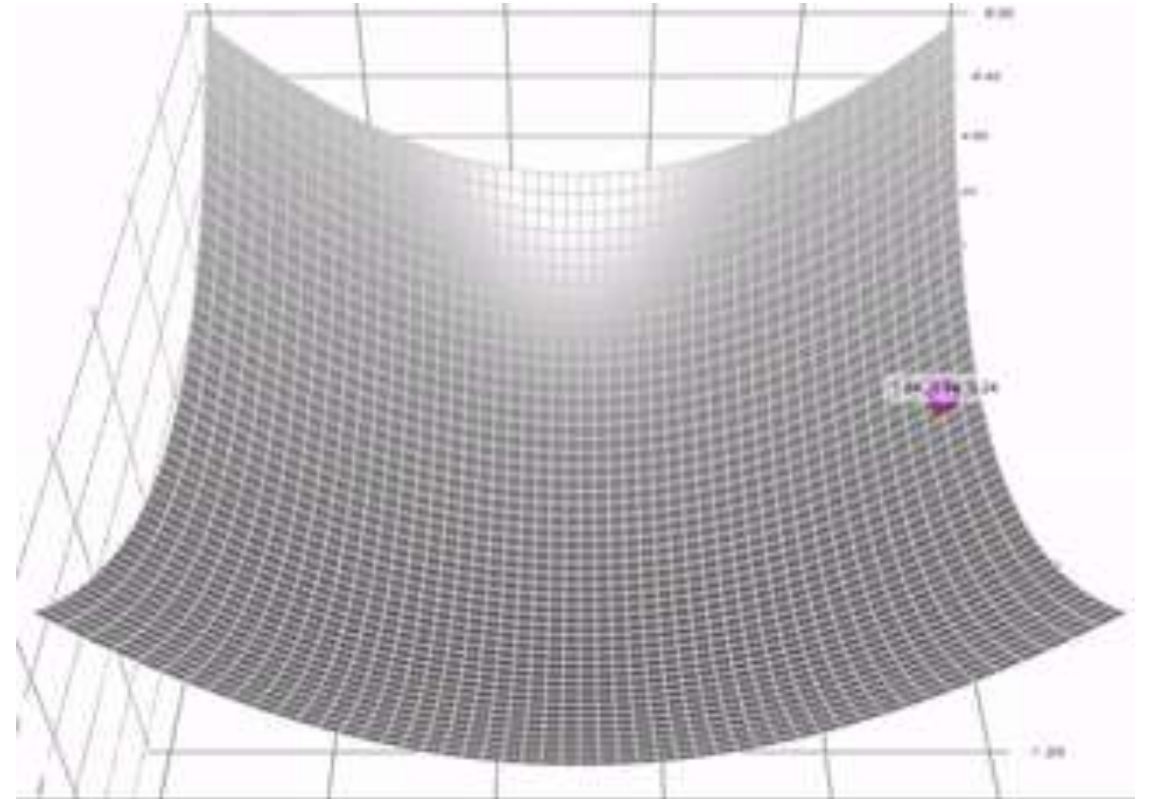
Idea

Like a heavy ball rolling downhill, its inertia keeps it moving

Rationale

Accelerate along directions that were previously good

Resist direction change when the gradient suddenly changes



Gradient descent with momentum

Parameter update

$$\theta_{t+1} = \theta_t - \rho_t \mathbf{m}_{t+1}$$

Where $\mathbf{m}_t$ is the momentum at time t

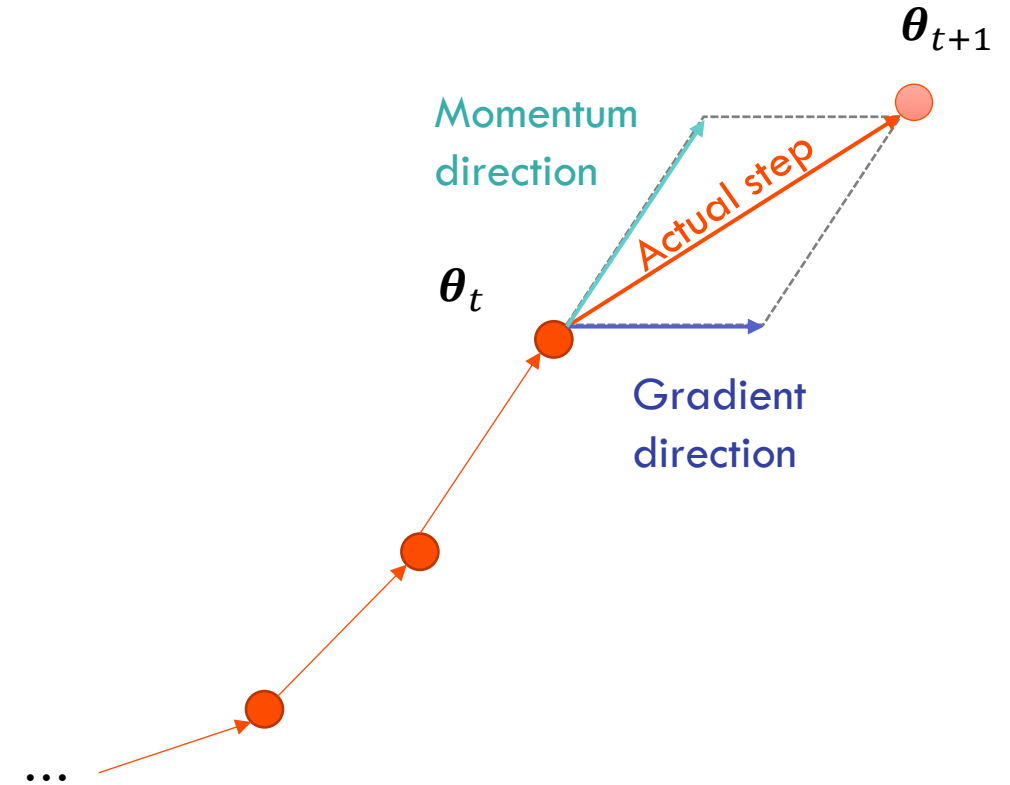
Momentum

Exponentially weighted moving average of the past gradients

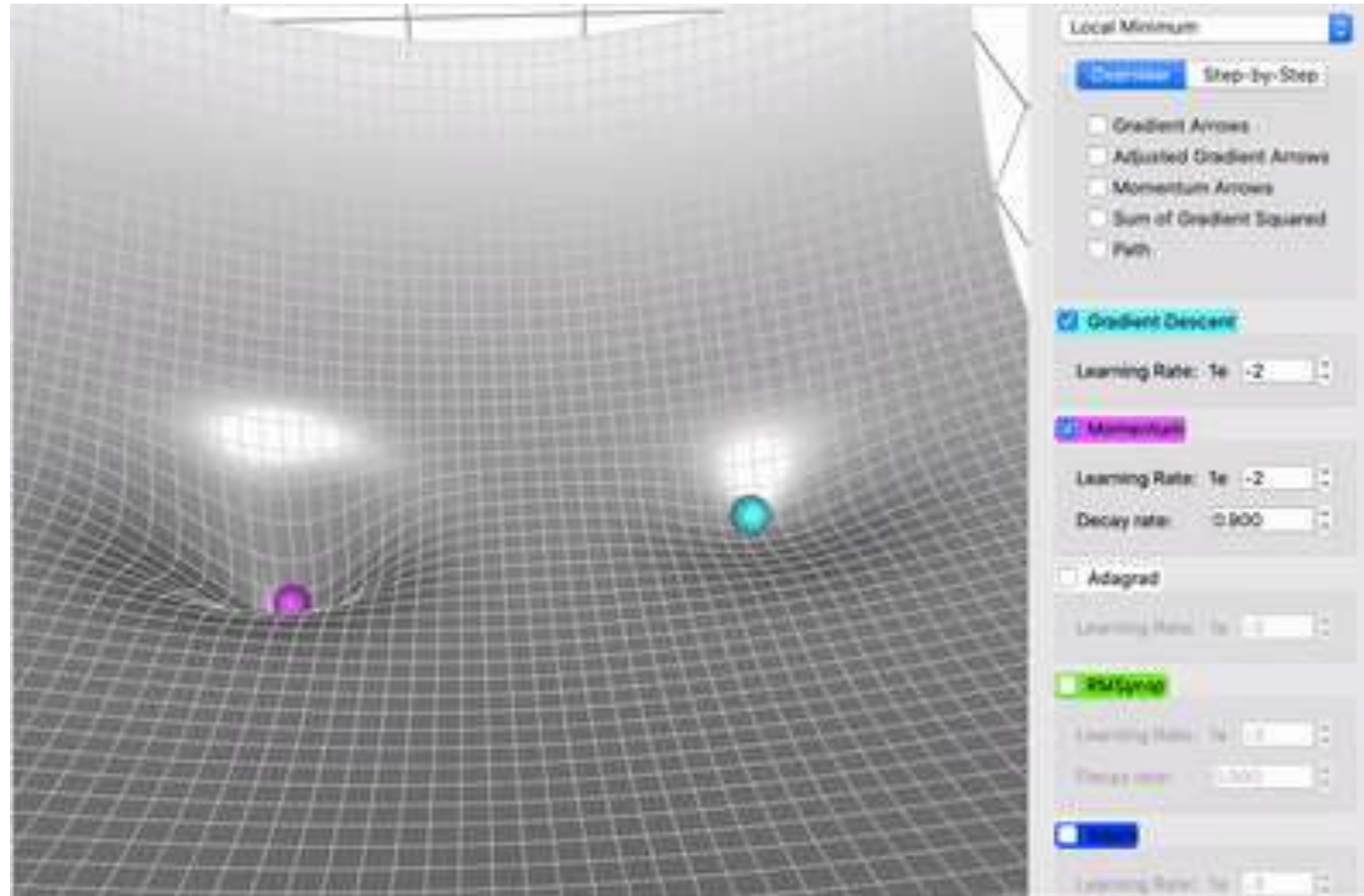
$$\mathbf{m}_{t+1} = \beta \mathbf{m}_t + \mathbf{g}_t$$

Where $0 < \beta < 1$ is the **decay rate**

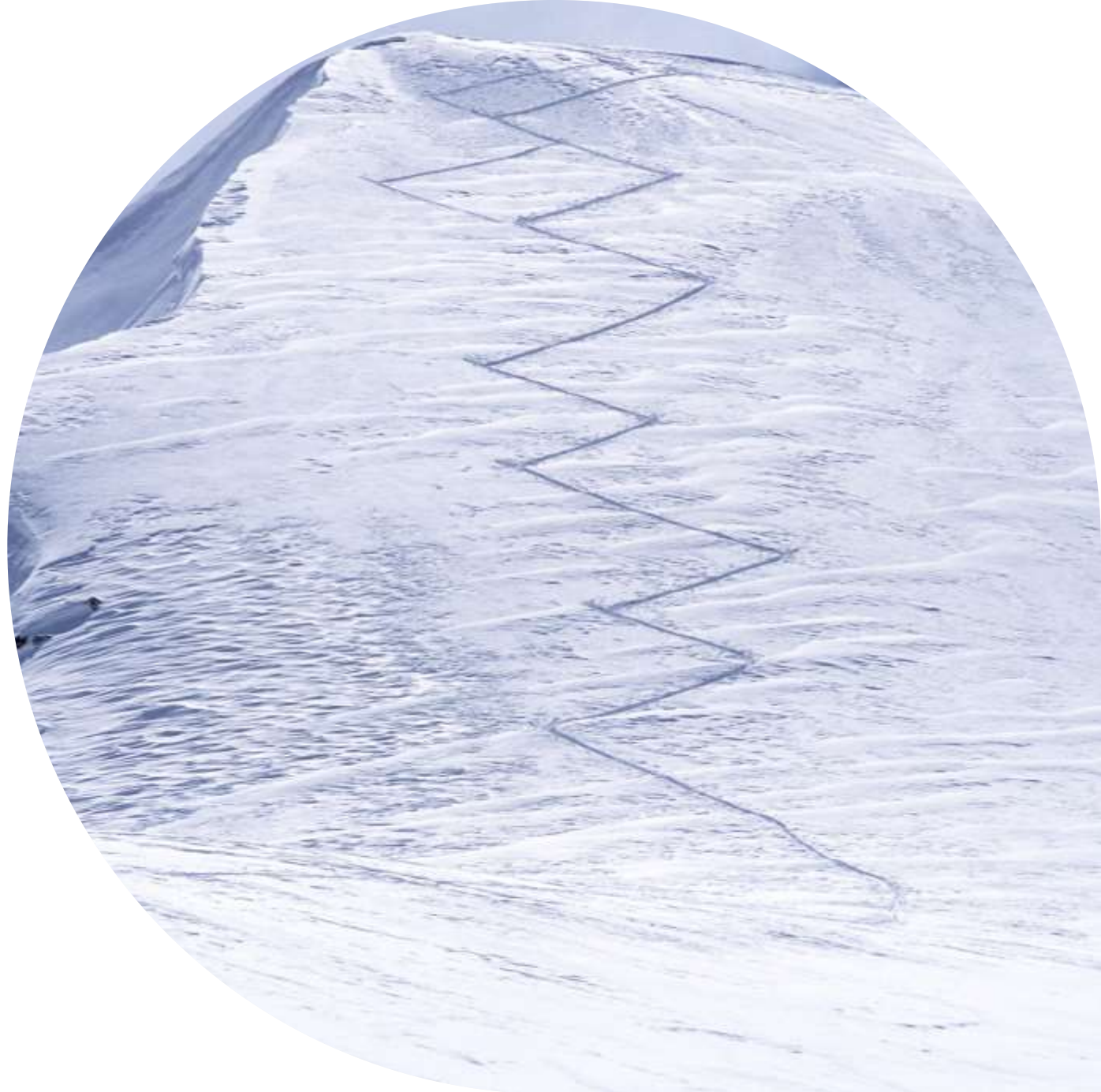
Typical default value: $\beta = 0.9$



Gradient descent with momentum



First-order algorithms
Preconditioned gradient
descent



Second-order methods

Newton's method

Uses the **optimal descent direction** given second-order information (i.e. curvature)

Descent direction

Given by the minimization of the local quadratic approximation (remember the Taylor expansion)

$$\mathbf{d}_t = -\mathbf{H}_t^{-1} \mathbf{g}_t$$

Step

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \rho_t \mathbf{H}_t^{-1} \mathbf{g}_t$$

Hessian adapts the step sizes for each direction according to local curvature

Preconditioned SGD

Accelerating SGD convergence

Adaptive learning rate methods

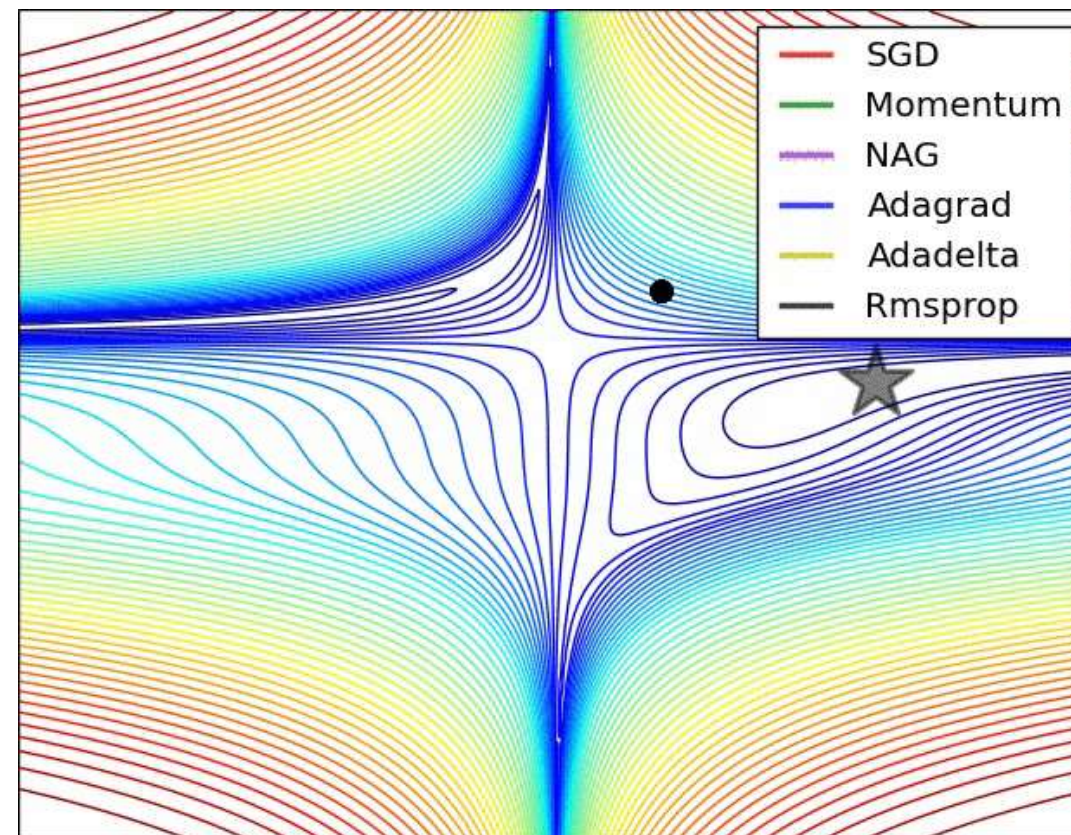
Adapting step sizes to local curvatures

(as a 2nd order method would do via the Hessian matrix)

$$\theta_{t+1} = \theta_t - \rho_t \mathbf{M}_t^{-1} \mathbf{g}_t$$

$\mathbf{M}_t$ is a **preconditioning matrix**

- Plays the role of the Hessian
- Computed from past gradients and updates



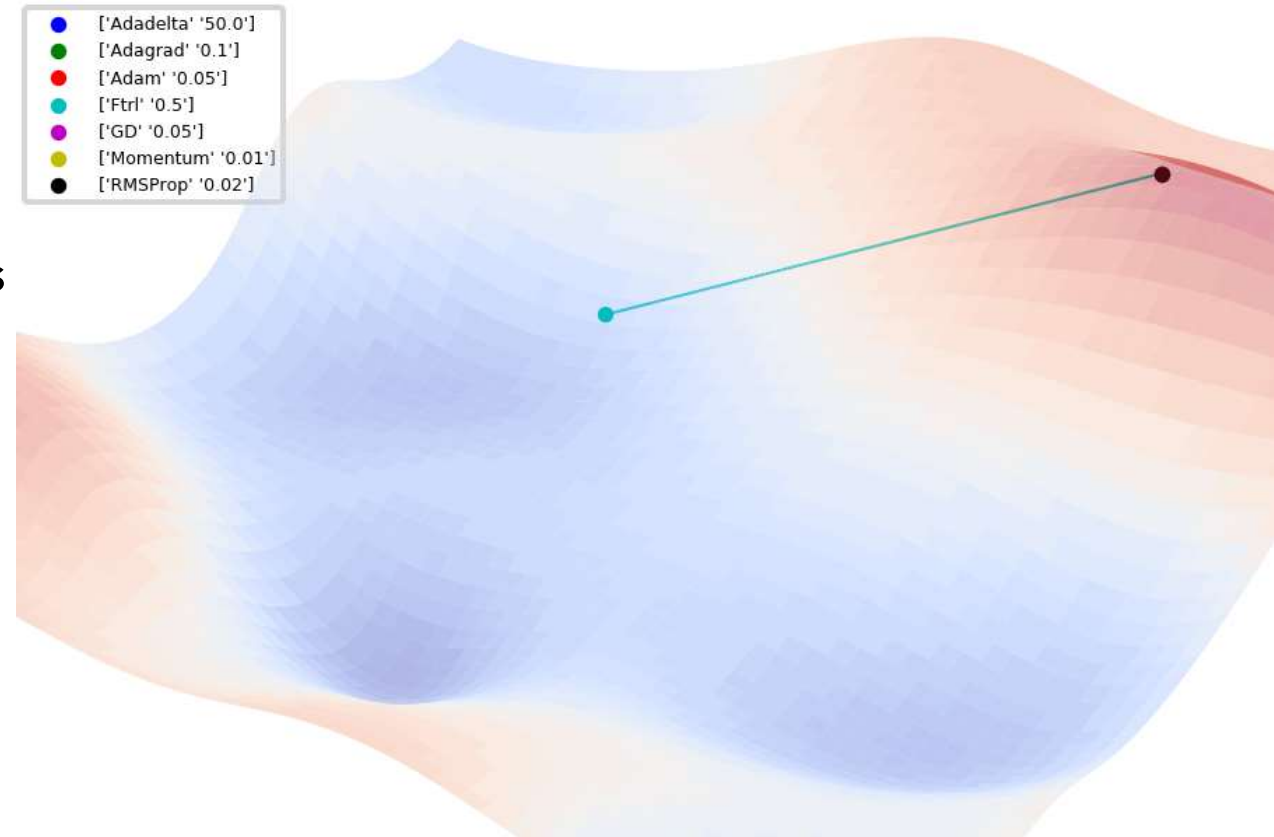
S. Ruder 2016, An overview of gradient descent optimization algorithms:
<https://ruder.io/optimizing-gradient-descent/>

Preconditioned SGD

Accelerating SGD convergence

Methods

- AdaGrad
 - Keeps track of past gradients via a squared sum term
 - Designed to handle gradients with many zeros
- RMSProp
 - Keeps track of past gradients via an Exponentially Weighted Moving Average (EWMA)
- AdaDelta
 - Keeps track of gradients + updates via EWMA
- Adam = RMSprop + momentum
 - **popular default choice**



[GitHub - Jaewan-Yun/optimizer-visualization: Visualize Tensorflow's optimizers.](#)